

IU Students

thesis

 Luong Quang Huy
 Thesis
 turnitin@hcmiu.edu.vn

Document Details

Submission ID**trn:oid::1:3572025423****Submission Date****May 17, 2026, 6:11 PM GMT+7****Download Date****May 17, 2026, 6:21 PM GMT+7****File Name****Thesis_LuongQuangHuy.pdf****File Size****1.8 MB****101 Pages****19,752 Words****117,441 Characters**

76% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



187 AI-generated only 76%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
School of Computer Science and Engineering**



**PATTERN-AWARE RETRIEVAL-AUGMENTED
GENERATION OF
MULTIPLE-CHOICE QUESTIONS FROM LECTURE
MATERIALS
USING LARGE LANGUAGE MODELS**

QuizGen: A Pattern-Aware RAG-Based MCQ Generation and Practice Platform

By

Luong Quang Huy

Student ID: ITITIU22076

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam

2026

**PATTERN-AWARE RETRIEVAL-AUGMENTED GENERATION OF
MULTIPLE-CHOICE QUESTIONS FROM LECTURE MATERIALS
USING LARGE LANGUAGE MODELS**

APPROVED BY:

Dang Tam Nhan, Supervisor

Committee Member

Committee Member

THESIS COMMITTEE

ACKNOWLEDGMENTS

First and foremost, I would like to express my sincere gratitude to my supervisor, **Dang Tam Nhan**, for the continuous guidance, constructive feedback, and encouragement throughout this thesis project. Your expertise in artificial intelligence and software engineering greatly shaped the direction of this work, and your patience during every revision cycle made this thesis possible.

I am grateful to the faculty of the **School of Computer Science and Engineering** at **International University – Vietnam National University Ho Chi Minh City** for providing an academic environment that encouraged independent research and critical thinking. The knowledge and skills I gained throughout my undergraduate years formed the foundation upon which this project was built.

I would also like to thank my fellow students and friends for the discussions, late-night study sessions, and mutual support. Special thanks to **Dai Hoang Viet** and **Nguyen Phuc Dat**, whose feedback during the early prototype stages helped identify issues I would not have noticed alone.

I am deeply thankful to my family for their unwavering love and encouragement throughout my studies. Their sacrifices and constant belief in me gave me the strength to persevere through the most challenging phases of this project.

Finally, I acknowledge the open-source community and the developers behind the tools and frameworks used in this thesis, including **FastAPI**, **Next.js**, **LangChain**, and the **Groq** and **Google Gemini** API platforms, without which building QuizGen at this scale would not have been feasible.

Luong Quang Huy

Ho Chi Minh City, May 6, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivating Scenario and Use Cases	2
1.3	Problem Statement	3
1.4	Scope and Objectives	4
1.5	Research Questions	6
1.6	Assumptions and Proposed Solution	7
1.7	Thesis Structure	8
2	Literature Review	10
2.1	Large Language Models in Education	10
2.2	Retrieval-Augmented Generation	12
2.3	Prompt Engineering and Few-Shot Learning	14
2.4	Bloom’s Taxonomy and Difficulty Control	16
2.5	Educational Assessment Requirements for MCQ Quality	17
2.6	Existing Question Generation Systems and Tools	18
2.7	Evaluation Approaches for Generated Questions	20
2.8	Research Gap	21
3	Methodology and System Design	23
3.1	Proposed Framework Overview	23
3.2	Design Requirements and Architectural Decisions	24
3.3	Document Processing and Chunking	25
3.4	Embedding and Retrieval	26

3.5	Exam Pattern Extraction	29
3.6	MCQ Generation Method	30
3.7	Prompt Assembly and Output Validation	32
3.8	Grounding and Hallucination Proxy	34
3.9	Quiz Practice and Confidence Trend	36
3.10	Database Design	37
3.11	Security and Reproducibility Considerations	38
4	Implementation and Results	40
4.1	Development Environment	40
4.2	Project Structure and Configuration	41
4.3	Backend Implementation	42
4.4	API Workflow Implementation	44
4.5	Frontend Implementation	45
4.6	User Interface State and Error Handling	48
4.7	Quiz Practice Implementation	49
4.8	Dashboard and Usage Implementation	51
4.9	Evaluation Dashboard Implementation	54
4.10	Testing and Demonstration Readiness	55
5	Discussion and Evaluation	58
5.1	Evaluation Design	58
5.2	Metrics	59
5.3	Baseline Rationale	62
5.4	Quantitative Results	63
5.5	Practical Interpretation of Results	65

5.6	Failure Analysis	66
5.7	Comparison with Related Work	68
5.8	Practical Implications for Lecturers and Students	69
5.9	Threats to Validity	70
5.10	Summary of Findings	71
6	Conclusion and Future Work	72
6.1	Conclusion	72
6.2	Contributions	73
6.3	Achievement of Objectives	73
6.4	Limitations	75
6.5	Future Work	75
	List of References	77
	Appendices	83
	Appendix A: Example Generated MCQs	83
	Appendix B: Evaluation Configuration	85
	Appendix C: Failure Analysis Table	87
	Appendix D: API Endpoint List	87
	Appendix E: Additional Screenshots	89
	Appendix F: Setup and Run Instructions	90

List of Tables

1	Comparison of existing question generation systems and QuizGen. . . .	20
2	Document processing configuration.	26
3	MCQ output schema.	32
4	Prompt output parsing and normalization behaviour.	34
5	Grounding status levels and semantic score thresholds.	35
6	Database entities and their roles.	37
7	QuizGen technology stack.	40
8	Repository structure used by the QuizGen implementation.	41
9	Backend API modules and services.	43
10	Representative API endpoints in the QuizGen workflow.	44
11	Implementation smoke-test summary.	56
12	Evaluation setup.	58
13	Evaluation metrics, definitions, and desired direction.	60
14	Core baseline comparison (mean $\pm$ std, 3 repeats, 10 topics).	64
15	Core baseline comparison (mean $\pm$ std, 3 repeats, 10 topics).	64
16	Representative failure cases from the evaluation.	67
17	Evidence-based comparison of QuizGen with related systems.	68
18	Achievement of thesis objectives.	74
19	Evaluation configuration summary.	85
20	Representative evaluation failures.	87
21	Main API endpoint groups.	87
22	Screenshot asset inventory.	89

List of Figures

1	Structure of the thesis.	9
2	Overall QuizGen pipeline from document upload to analytics.	23
3	RAG retrieval sequence from query to prompt context.	27
4	Exam pattern and difficulty configuration step in the QuizGen wizard. . .	30
5	Quiz practice and confidence trend sequence.	37
6	Simplified entity–relationship overview of the QuizGen database.	38
7	Source step: document upload and selection in the QuizGen wizard. . .	46
8	Generate step: quiz title assignment before submitting the generation request.	47
9	Review step: generated quiz with grounding status, Bloom labels, and difficulty indicators.	48
10	Quiz practice interface showing a question with answer options and the elapsed timer.	50
11	Attempt review page showing per-question correctness, correct answers, explanations, and Bloom levels.	51
12	Generation history panel with inline quiz rename functionality.	52
13	Confidence trend chart showing attempt score percentage over time across stored user attempts.	53
14	Usage dashboard showing per-provider API call statistics and token consumption.	54
15	Evaluation dashboard showing the latest baseline comparison results with mean and standard deviation values.	55

ABSTRACT

Large language models (LLMs) have opened practical opportunities for automating parts of educational content creation. In assessment, however, generating high-quality multiple-choice questions (MCQs) remains difficult. Lecturers still need questions that are tied to the taught material, have a clear answer key, use plausible distractors, and fit the expected exam style, difficulty distribution, and cognitive level. Generic LLM outputs can be fluent, but they may also hallucinate unsupported content or ignore local assessment patterns.

In this thesis, I propose **QuizGen**, a pattern-aware retrieval-augmented generation (RAG) platform for generating grounded multiple-choice questions from lecture materials. QuizGen addresses the limitations of generic generation by combining lecture-material retrieval, exam-pattern conditioning, and Bloom's taxonomy control within a unified web-based workflow.

QuizGen works as a multi-stage pipeline. Uploaded lecture documents in PDF, DOCX, and PPTX formats are extracted and split into overlapping word chunks with a chunk size of 2,000 words and an overlap of 200 words. These chunks are embedded through the configured embedding provider and stored for semantic retrieval. When a lecturer requests a quiz, cosine similarity retrieves the most relevant chunks and injects them into a structured prompt, optionally alongside an exam-pattern template extracted from past exams. The prompt also encodes the target Bloom's taxonomy level and difficulty distribution, so the LLM returns questions with answer options, the correct answer, an explanation, a Bloom label, and a difficulty rating. Grounding evaluation is available as an explicit review step via the evaluation endpoint, which measures semantic overlap with the source document and flags each item as *well grounded*, *partially*

grounded, or poorly grounded. After review, the questions become a playable quiz; attempts, scores, timings, and Bloom-level accuracy are stored and shown in dashboard views, with the current backend dashboard aggregation limitation documented in the testing notes.

Evaluation on ten topics in English and Vietnamese, with three repeated runs and six questions per topic, shows that RAG improves semantic grounding over vanilla LLM generation. Exam-pattern conditioning further improves Bloom-level distribution alignment. Among the tested baselines, the full system achieves the highest LLM judge quality score while returning the full question set in every run.

Several limitations remain. The evaluation dataset is thesis-scale and may not generalize to all domains or languages. The grounding score is a semantic proxy, not a guarantee of factual correctness, so lecturer review is still required before formal exam use.

Keywords: multiple-choice question generation, retrieval-augmented generation, large language models, Bloom's taxonomy, exam-pattern conditioning, educational technology.

CHAPTER I

INTRODUCTION

1.1 Background

Artificial intelligence has increasingly become part of educational technology, especially in tasks that involve reading, summarizing, tutoring support, and content preparation. Large language models (LLMs) are particularly relevant because they can produce fluent natural-language outputs from relatively small prompts. For assessment, this capability is useful but also risky: many courses need frequent quizzes and exam-style questions, yet those questions must still reflect what was actually taught.

Multiple-choice questions (MCQs) are widely used because they are structured, easy to grade automatically, and suitable for both formative practice and summative examination. A well-designed MCQ can test factual recall, conceptual understanding, and higher-order reasoning. For students, MCQs also provide a practical way to review course knowledge and identify weaknesses before an exam.

However, producing good MCQs is not a simple task. Each question must be relevant to the lecture materials and have one clearly correct answer. Distractors should be plausible without being misleading, and the overall difficulty level should match what students have actually been taught. When a course contains many lecture slides or reading materials, producing a large number of questions by hand becomes time-consuming. Question quality also varies with the lecturer's available time, experience, and familiarity with the intended assessment style.

LLMs can generate questions quickly, but direct generation without source

control may produce questions that are fluent but unsupported by the actual teaching materials. In an academic setting, generated questions should be traceable to the content that students have learned. For that reason, automation alone is not enough; the generation process also needs grounding in lecture materials and alignment with the desired exam style.

These observations motivate the development of QuizGen, a pattern-aware retrieval-augmented generation system for creating multiple-choice questions from lecture materials. I designed it to help lecturers generate reviewable questions and to give students a way to practice those questions inside a web-based learning workflow.

1.2 Motivating Scenario and Use Cases

A typical motivating scenario is a lecturer preparing a weekly practice quiz after finishing a lecture topic. The lecturer may already have a PDF slide deck, a DOCX handout, or a PPTX lecture file, but still needs to turn that material into questions that match the style of the course. Preparing all questions manually requires repeated reading, selection of assessable concepts, distractor drafting, answer-key checking, and revision of weak items. The work is pedagogically important, but much of it is also repetitive.

From the student's perspective, the problem appears differently. Students often want more practice questions than the lecturer can reasonably prepare by hand. They also need quick feedback after answering: whether their selected answer is correct, why the correct option is correct, and which area of the course they should review next. A static list of generated questions is useful, but it becomes more valuable when it is connected to a practice interface, attempt history, and progress visualization.

QuizGen targets three concrete use cases. The first is *lecturer-assisted quiz drafting*: a lecturer uploads lecture materials, optionally selects a saved exam pattern,

generates a set of MCQs, and reviews grounding signals before using the questions. The second is *student self-practice*: a student opens a saved quiz, answers the questions, and reviews explanations immediately after submission. The third is *course analytics*: repeated attempts are collected so the application can show score trends and Bloom-level weaknesses over time.

These use cases clarify a design principle that runs through the whole thesis. The goal is not to replace the lecturer as the final authority on assessment quality. Instead it is to reduce the mechanical effort needed to produce draft questions, while keeping enough traceability for the lecturer to make an informed decision. For this reason QuizGen pairs source retrieval with a grounding evaluator and a review interface, rather than exposing raw LLM output as a finished exam artifact.

1.3 Problem Statement

Current MCQ creation still depends heavily on manual effort. In a typical course, lecturers need to review lecture materials, identify important concepts, decide the intended difficulty level, write the question stem, prepare answer options, and verify that the final question is correct. This process is repetitive and can become burdensome when many quizzes are needed across different topics.

Recent LLMs offer a promising solution because they can produce natural-language questions with little input. Nevertheless, using an LLM directly for academic question generation introduces several problems.

First, generated questions may not be grounded in the uploaded lecture materials. A model may rely on its general knowledge and generate content that appears reasonable but is not actually covered in the course. In educational assessment, this is a serious issue because students should be tested on the intended learning materials, not on unsupported

external content.

Second, generic LLM outputs may not follow a specific exam pattern. In practice, each course may have its own style of questions, answer-option structure, and difficulty distribution. Without additional conditioning, generated questions may be inconsistent with the style that lecturers expect.

Third, it is difficult to control the cognitive level of generated questions. Educational design often uses Bloom's taxonomy to distinguish between different levels of learning, such as remembering, understanding, applying, and analyzing. If this distribution is not controlled, the generated quiz may contain too many simple recall questions or may fail to match the intended learning objectives.

Fourth, many question generation approaches stop at producing question text. They do not provide a complete workflow for students to attempt the generated quiz, review their answers, track scores, or observe learning progress over time. As a result, the generated questions are less useful as part of a practical learning system.

The main problem addressed in this thesis is how to design and implement a system that can generate exam-style MCQs grounded in uploaded lecture materials, aligned with a target question pattern, and usable within an interactive quiz practice workflow.

1.4 Scope and Objectives

The main objective is to design, implement, and evaluate QuizGen as a web-based system for generating grounded and pattern-aware multiple-choice questions from lecture materials.

My focus is on a complete workflow rather than on a single algorithm. A lecturer can upload lecture materials, configure or select a question pattern, generate MCQs,

review the generated results, and publish them as a playable quiz. Students attempt the quiz, view their results, and track performance through the application.

The specific objectives are:

- To build a document processing pipeline that supports lecture materials in PDF, DOCX, and PPTX formats.
- To apply retrieval-augmented generation so that generated questions are based on relevant content chunks retrieved from the uploaded materials.
- To incorporate exam-pattern conditioning so that generated questions can better follow a desired question style and structure.
- To generate structured MCQ outputs, including question text, answer options, correct answer, explanation, difficulty level, and Bloom's taxonomy label.
- To provide a grounding evaluation mechanism that helps identify whether a generated question is strongly or weakly supported by the source materials.
- To implement a web application that supports quiz generation, review, practice, attempt history, and performance visualization.
- To evaluate the system using thesis-scale experiments and compare the full system with baseline generation approaches.

Within the scope of this thesis, QuizGen is limited to multiple-choice question generation. I built it as a local thesis-scale prototype rather than a high-concurrency production platform. The current implementation uses a SQLite database and integrates LLM-based services for embedding, retrieval, and generation. Evaluation is conducted on a limited dataset covering English and Vietnamese topics.

Several aspects are outside the current scope. The system does not replace lecturer review for formal examinations. It does not guarantee that every generated

question is fully correct, even when a grounding score is provided. The grounding score is treated as a support signal rather than a perfect factuality detector. Large-scale classroom deployment, production database migration, and human expert evaluation are left for future work.

1.5 Research Questions

The thesis is guided by three research questions that correspond to the main technical components of QuizGen.

1. **RQ1: Does retrieval-augmented generation improve the source grounding of generated MCQs compared with direct LLM prompting?** This question examines whether retrieving lecture chunks before generation makes the generated questions more semantically aligned with the uploaded source material.
2. **RQ2: Does exam-pattern conditioning improve alignment with the intended Bloom-level distribution?** This question evaluates whether few-shot examples and explicit distribution instructions can steer the LLM toward a desired cognitive-level mix without model fine-tuning.
3. **RQ3: Can the generated questions be integrated into a usable web-based practice workflow?** This question focuses on the practical system outcome: upload, generation, review, quiz attempt submission, attempt review, usage tracking, and dashboard analytics.

These questions shape the evaluation design in Chapter V. RQ1 is assessed through semantic grounding and retrieval metrics. RQ2 is assessed through Bloom KL divergence and failure analysis. RQ3 is assessed through the implemented end-to-

end web workflow, screenshots, API coverage, and the manual smoke-test evidence summarized in Chapter IV.

1.6 Assumptions and Proposed Solution

The work assumes that uploaded lecture materials are relevant, readable, and sufficiently informative for question generation. It also assumes that lecturers or authorized users will review generated questions before using them in formal assessment. The intended application context is a university or classroom environment where lecturers need practice or exam-style questions, and students need additional opportunities for self-practice.

To address the identified problems, I propose QuizGen, a pattern-aware retrieval-augmented generation system. It combines semantic retrieval and exam-pattern conditioning with Bloom-level control. A grounding evaluation component and a quiz practice interface complete the workflow inside a single web application.

The workflow begins with document upload and text extraction. Extracted lecture content is divided into overlapping chunks, and each chunk is converted into a vector embedding. When a quiz is generated, the backend retrieves the most relevant chunks using semantic similarity and inserts them into the generation prompt. The prompt may also include an exam pattern derived from sample questions, together with the desired difficulty and Bloom-level distribution.

Each generated MCQ carries a question stem, four answer options, the correct answer, an explanation, a difficulty label, and a Bloom's taxonomy label. A grounding evaluator then scores how well each question is supported by the source material. Reviewed questions become a playable quiz. As students attempt the quiz, their answers, scores, timing, and per-Bloom correctness are stored and surfaced through the dashboard.

Through this design, QuizGen aims to reduce the manual effort required for MCQ preparation while keeping the generated questions connected to lecture materials and usable in a practical learning workflow.

1.7 Thesis Structure

The remainder of the thesis is organized into five further chapters. After this introduction, Chapter II reviews related work on large language models in education, retrieval-augmented generation, prompt engineering, and Bloom's taxonomy, along with a survey of existing question generation systems and the research gap that motivates QuizGen.

Chapters III and IV form the technical core. In Chapter III I present the methodology and system design, covering the proposed pipeline, document processing, embedding and retrieval, exam-pattern extraction, the MCQ generation method, grounding evaluation, and the database schema. Chapter IV then describes the implementation: development environment, backend and frontend modules, the quiz practice workflow, and the dashboards for performance, usage, and evaluation.

The final two chapters analyze and conclude the work. Chapter V evaluates QuizGen against three baselines, reports quantitative results, examines failure cases, and discusses threats to validity. Chapter VI summarizes the contributions and outlines future improvements.

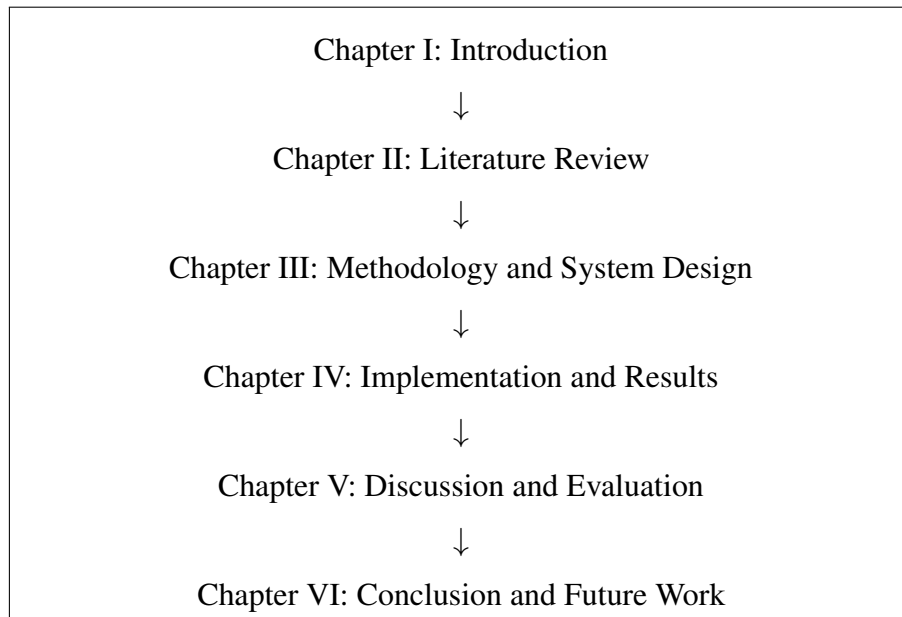


Figure 1: Structure of the thesis.

CHAPTER II

LITERATURE REVIEW

2.1 Large Language Models in Education

Large language models (LLMs) are neural networks trained on massive text corpora that can produce fluent, contextually appropriate natural language across a wide range of tasks [1]. In education, LLMs have attracted growing interest as tools for content summarization, intelligent tutoring, automated grading, and question generation. Xu et al. [2] provide a comprehensive survey of LLM applications in education, organising the landscape around four pillars: student assistance, teacher assistance, adaptive learning, and educational assessment. The survey concludes that LLMs “perform outstandingly in improving teaching efficiency and interactivity” but also face substantial challenges in data privacy, model transparency, and domain-specific factual accuracy.

A parallel survey by Xu et al. [3] extends this analysis by cataloguing benchmarks and evaluation datasets specific to educational AI. The authors observe that most existing systems focus on general-purpose question answering and reading comprehension rather than structured assessment generation tied to lecture materials. This gap motivates research that places LLM generation within a curriculum-grounded pipeline.

Empirical studies confirm both the promise and the limitations of LLMs for classroom use. A systematic review published in *Frontiers in Education* [4] examined 47 studies and found that ChatGPT-based tools improve student engagement and provide useful formative feedback, but show uneven performance across subjects and tend to reduce critical-thinking demands when used without appropriate scaffolding. A case

study by Pham et al. [5] at IEEE BigData 2023 evaluated ChatGPT (GPT-3.5-turbo) for question generation from passage context and demonstrated that prompt engineering alone, without retrieval grounding, can produce competitive outputs in well-scoped domains.

Hallucination remains the most critical risk for educational deployment. Danyaro et al. [6] identify hallucination as a fundamental reliability barrier, noting that LLMs may produce plausible but factually incorrect statements, particularly in specialised academic domains. The authors recommend knowledge injection, human review loops, and source-grounded generation as primary mitigation strategies. These findings directly motivate the retrieval-augmented approach adopted in this thesis.

The recent LLM education landscape also continues a longer history of adaptive educational technology. Intelligent tutoring systems (ITS) were already being studied in the 1980s as systems that combine a domain model, a student model, and pedagogical feedback rules. Anderson, Boyle, and Reiser [13] describe computer tutors for geometry proofs and LISP programming that were grounded in cognitive theory rather than in general text generation. This lineage is important because modern LLM tutors do not introduce the educational goal of adaptive support from scratch; instead, they provide a new language-interface layer on top of a much older aim: diagnosing learner needs and responding with timely, personalised guidance.

Contemporary products demonstrate how this lineage is being reinterpreted with generative models. Khan Academy's Khanmigo is presented as an AI-powered tutor and teaching assistant that guides students rather than simply giving direct answers, and also offers teacher-facing tools such as lesson planning, rubrics, and exit tickets [15]. Duolingo Max similarly uses GPT-4-powered features for language learning, including interactive roleplay and answer explanation features that give learners feedback inside

the practice flow [16]. These systems show that LLMs are increasingly embedded inside complete learning products, not merely used as standalone chatbots.

Automated assessment has an even older technical history. Automated essay scoring systems such as Project Essay Grade (PEG) were proposed in the 1960s and later evolved into large-scale writing assessment tools [14]. The relevance to MCQ generation is methodological: automated educational systems must be evaluated not only for linguistic fluency but also for validity, reliability, and alignment with the intended assessment construct. QuizGen therefore treats generation quality as a combination of source grounding, cognitive-level alignment, answer correctness, and post-generation review rather than as a pure text generation problem.

2.2 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [7] is a framework that augments a generative language model with a retrieval component. Given an input query, the retrieval module selects relevant passages from an external knowledge store; these passages are then concatenated with the query and passed to the generator as additional context. Lewis et al. [7] demonstrate that this architecture outperforms closed-book generation on knowledge-intensive NLP tasks such as open-domain question answering and fact verification, because the retrieved context constrains the generator to produce responses supported by authoritative source material.

In the context of educational question generation, RAG addresses the core weakness of vanilla LLM prompting: the tendency to hallucinate content not covered in the course. By indexing lecture materials and retrieving the most relevant chunks at generation time, the system ensures that each MCQ is anchored to a specific segment of the uploaded document. This traceability is particularly valuable in academic settings

where assessment questions must be justifiable from the course syllabus.

Evaluating RAG systems introduces its own methodological challenges. Es et al. [8] propose RAGAS, a reference-free evaluation framework that measures retrieval quality (context recall and precision) and generation quality (faithfulness and answer relevance) without requiring gold-standard annotations. RAGAS treats faithfulness, defined as the degree to which generated statements are entailed by the retrieved context, as the primary grounding signal. This thesis adopts a semantic grounding score inspired by this principle, adapted for the MCQ generation domain where ground-truth answer keys are available but annotated question quality is not.

RAG depends first on the quality of retrieval. Dense Passage Retrieval (DPR) showed that open-domain passage retrieval can be implemented with learned dense representations using a dual-encoder architecture, rather than relying only on sparse lexical matching such as BM25 [17, 18]. This shift is relevant to lecture-slide retrieval because students and lecturers may request a topic using vocabulary that differs from the exact wording in the source document. Dense embeddings provide a way to retrieve passages by semantic similarity, which is the basis of the cosine-similarity search used in QuizGen.

Sparse and dense retrieval are complementary rather than mutually exclusive. BM25 remains useful for exact terms, abbreviations, and contrastive wording, while dense retrieval can recover semantically related passages that do not share the same surface vocabulary. Reciprocal Rank Fusion provides a simple way to combine multiple ranked lists without training a new ranker [19]. Neural passage reranking, such as BERT-based reranking, further motivates a second-stage model that scores candidate passages after an initial retrieval step [20]. These methods inform the post-review retrieval ablation, although QuizGen keeps the dense retriever as the default implementation for

the main thesis comparison.

Subsequent RAG variants show different ways to connect retrieved evidence to the generator. Fusion-in-Decoder (FiD) encodes retrieved passages separately and fuses them during decoding, improving open-domain question answering when many passages must be considered [21]. REPLUG takes a more black-box-friendly approach by prepending retrieved documents to the input of an existing language model and using the language model to supervise retriever improvement [22]. QuizGen follows the simpler black-box pattern because the implemented system calls hosted LLM APIs rather than training or modifying model internals.

RAG also introduces failure modes that are important for educational use. The retriever may select chunks that are semantically close but pedagogically irrelevant, may miss a small but essential definition, or may retrieve too many loosely related chunks and dilute the useful context. Even when the right chunk is retrieved, the generator can still ignore part of the evidence or combine it with unsupported prior knowledge. Evaluation frameworks such as RAGAS [8] and ARES [23] respond to this problem by separating dimensions such as context relevance, faithfulness, and answer relevance. QuizGen uses a smaller thesis-scale grounding proxy, but the design follows the same principle: source support must be measured separately from general answer fluency.

2.3 Prompt Engineering and Few-Shot Learning

Prompt engineering refers to the systematic design of input instructions to elicit desired behaviours from a pre-trained language model without updating its parameters [10]. It offers a lightweight alternative to fine-tuning, requiring no labelled training data and no GPU-intensive optimisation, making it practical for domain-specific applications where training data is scarce or expensive to collect.

White et al. [9] introduce a prompt pattern catalog that formalises common prompting strategies as reusable templates. Relevant patterns for this thesis include the *Persona pattern* (assigning the model a role such as “You are a university lecturer in Computer Science”), the *Template pattern* (providing output schemas such as JSON structures for MCQ fields), and the *Few-shot example pattern* (including representative questions in the prompt to demonstrate the target style). Together, these patterns form the basis of exam-pattern conditioning in QuizGen.

Few-shot learning [1] allows a model to generalise from a small number of in-context examples. In the QuizGen pipeline, sample questions extracted from past exams serve as few-shot demonstrations of the desired question style, difficulty level, and answer-option structure. This conditioning mechanism does not require access to model weights and is therefore compatible with API-based LLM providers. A detailed description of the prompt structure and conditioning logic is provided in Chapter III.

Prompting research also distinguishes between surface formatting control and reasoning control. Chain-of-thought prompting asks a model to use intermediate reasoning steps before giving an answer, and Wei et al. [26] show that this can improve performance on arithmetic, commonsense, and symbolic reasoning tasks for sufficiently large models. Self-consistency extends this idea by sampling multiple reasoning paths and selecting the answer that appears most consistently across those paths [27]. QuizGen does not expose hidden reasoning traces to students, but these methods justify why prompt structure matters for higher-order questions: application and analysis items often require the model to connect concepts rather than merely restate definitions.

The implemented prompt therefore uses direct constraints instead of free conversation. It assigns the model an exam-generator role, supplies retrieved source material, optionally adds lecturer-provided examples, and requires a JSON schema containing

the question, four options, answer, explanation, difficulty, and Bloom label. This design combines the prompt-pattern view of White et al. [9] with the practical need for structured backend parsing.

2.4 Bloom's Taxonomy and Difficulty Control

Bloom's taxonomy [11] organises cognitive learning objectives into six hierarchical levels: *Remember*, *Understand*, *Apply*, *Analyze*, *Evaluate*, and *Create*. Originally published in 1956 and revised in 2001, the framework remains a standard tool in educational design for specifying and assessing the cognitive demands of learning activities.

The revised taxonomy by Anderson and Krathwohl reframes the cognitive process dimension using verb forms—remember, understand, apply, analyze, evaluate, and create—and separates this from the knowledge dimension [12]. This revision is better suited to automated generation than the original noun-based hierarchy because prompts can ask for observable question behaviours, such as “explain”, “apply”, or “analyze”. QuizGen adopts the revised labels in the prompt and database schema, while retaining Bloom's taxonomy as the educational framework.

In the context of automated question generation, Bloom's taxonomy serves two distinct functions. First, it provides a vocabulary for *controlling* the cognitive level of generated questions: by encoding a target Bloom label in the generation prompt, the system can steer the LLM toward questions that require recall, application, or analysis. Second, it provides a metric for *evaluating* the cognitive balance of a generated question set: the Kullback–Leibler (KL) divergence between the target Bloom distribution and the observed distribution of generated labels measures how closely the system honours the intended difficulty mix.

Research in neural question generation confirms the practical relevance of this

control mechanism. A survey of recent advances in neural question generation by Pan et al. [24] identifies cognitive-level targeting as an open challenge, noting that most existing systems generate lower-order recall questions by default and require explicit conditioning to produce higher-order analytical questions. QuizGen addresses this limitation by including the target Bloom level and difficulty distribution as explicit prompt parameters, and by measuring KL divergence as a primary evaluation metric.

Automatic question generation research reaches the same conclusion from a broader educational perspective. Kurdi et al. [25] review automatic question generation for educational purposes and show that the field spans rule-based, semantic, and neural methods, but still requires attention to pedagogical validity and question type. For this thesis, Bloom labels are not treated as guaranteed truth simply because the LLM outputs them. They are used as controllable metadata that can be checked in aggregate against a target distribution and reviewed manually for high-stakes use.

2.5 Educational Assessment Requirements for MCQ Quality

Although automatic question generation is often evaluated with language similarity or fluency metrics, educational assessment requires a broader definition of quality. A generated MCQ is useful only if it satisfies both linguistic and pedagogical requirements. The question stem should be clear, unambiguous, and focused on one assessable concept. The answer options should include exactly one correct answer, and the distractors should be plausible enough to reveal misconceptions rather than being obviously incorrect. The explanation should connect the correct answer back to the learning material so that the question can support formative feedback.

For lecture-grounded generation, relevance is not sufficient. A question may be relevant to the general subject area while still being unsupported by the uploaded

lecture slides. For example, an operating systems quiz about CPU scheduling should not introduce a scheduling algorithm that was not covered in the specific lecture material used for the course. This distinction is central to QuizGen: the system treats source support as a separate dimension from natural-language quality.

Another requirement is alignment with the intended assessment pattern. Many courses develop recognizable styles over time, such as short definition questions, scenario-based application questions, or numerical reasoning questions. A generic LLM can produce grammatically correct MCQs, but the questions may feel inconsistent with the lecturer's existing exam style. Few-shot pattern conditioning provides a practical way to express this style requirement without collecting a large training dataset.

Finally, MCQs used for practice should be actionable after submission. Students benefit when the system records their attempt, shows explanations, and helps them identify which cognitive levels remain weak. This requirement extends the scope of question generation from a one-time text generation task into a learning application. It is the reason this thesis includes attempt storage, confidence trend visualization, and Bloom-level analytics as part of the implemented system.

2.6 Existing Question Generation Systems and Tools

A number of systems have been proposed or deployed for automatic question generation. A systematic literature review of transformer- and LLM-based MCQ generation methods published at IEEE in 2025 [29] identifies fine-tuning and prompt engineering as the two dominant paradigms, and notes that most systems optimise for generation fluency but do not evaluate grounding or cognitive-level alignment.

The **E-QGen** system [31] generates questions from lecture abstracts using GPT-3.5 and GPT-4 as backends. E-QGen demonstrates that structuring prompts with role,

task, context, and format components improves output relevance, but the system does not retrieve source evidence and provides no grounding evaluation. The **BUE AQG System** [32] combines BERT-based keyword extraction, T5-based question generation, and GPT-based answer generation, achieving an 80% lecturer satisfaction rate in user studies. However, it does not support exam-pattern conditioning or quiz practice workflows. A study presented at COLING 2025 [30] applied prompt engineering to university regulation documents and demonstrated effective MCQ generation, but similarly lacked integration with a practice interface or performance analytics.

Commercial tools such as the **GPT Quiz Generator for Google Forms** provide practical MCQ generation across multiple languages using GPT-4o mini, but offer no retrieval grounding, no Bloom-level control, and no student attempt tracking. **Generic ChatGPT prompting**, used as the vanilla baseline in this thesis, represents the simplest deployment scenario: direct LLM generation without any source retrieval or pattern conditioning.

Two additional commercial systems illustrate the same trade-off. Quillionz advertises AI-powered quiz and assessment generation from user-provided content, with exports to formats such as PDF, TXT, DOC, and QTI, and states that its tool works best on structured factual educational content [33]. Questgen supports quiz generation from text, images, PDFs, videos, and URLs, and advertises MCQ, true/false, fill-in-the-blank, higher-order, and Bloom-taxonomy quiz generation with export formats such as QTI, Moodle XML, CSV, and PDF [34]. These tools are useful reference points because they emphasise productivity for teachers, but their public product descriptions do not show an end-to-end research evaluation of retrieval grounding, exam-pattern conditioning, Bloom KL divergence, and student attempt analytics in one pipeline.

Table 1 summarises the key features of these systems relative to QuizGen.

Table 1: Comparison of existing question generation systems and QuizGen.

System	Source grounding	Exam pattern	Bloom control	Quiz practice	Performance analytics
Generic ChatGPT prompting	×	×	×	×	×
GPT Quiz Generator (Forms)	×	×	×	×	×
E-QGen [31]	×	Partial	×	×	×
BUE AQG [32]	×	×	×	×	×
COLING 2025 [30]	×	×	×	×	×
Quillionz [33]	Partial	×	Partial	×	×
Questgen [34]	Partial	×	✓	Partial	×
QuizGen (this thesis)	✓	✓	✓	✓	✓

2.7 Evaluation Approaches for Generated Questions

Evaluating generated MCQs is more difficult than evaluating ordinary classification output because several properties must be considered at the same time. A question may be grammatical but unsupported by the source material, grounded but too easy, or aligned with the intended Bloom level but phrased with weak distractors. Therefore, this thesis uses a multi-metric evaluation strategy instead of relying on a single score.

Traditional text-generation metrics such as BLEU and ROUGE compare a generated sentence with one or more reference sentences. These metrics are useful when there is a known target answer, but they are less suitable for open-ended MCQ generation because many valid questions can be produced from the same lecture paragraph. A low lexical overlap score does not necessarily mean the generated question is poor, and a high lexical overlap score does not guarantee that the answer key is correct.

For this reason, QuizGen evaluates generated questions through task-specific signals. Retrieval quality is measured using Recall@ k and MRR against annotated source chunks. Source grounding is measured with normalized cosine similarity between

generated question content and source material. Bloom alignment is measured with KL divergence between the target and observed Bloom distributions. Overall quality is estimated by an LLM judge using a rubric that includes relevance, correctness, clarity, and groundedness.

This combination has two advantages for a thesis-scale evaluation. First, it separates the effects of retrieval, pattern conditioning, and generation quality, making it possible to explain why one baseline performs better than another. Second, it acknowledges the limits of automation: the metrics support comparison and error analysis, but they do not remove the need for lecturer review before high-stakes use.

2.8 Research Gap

The preceding review reveals a consistent pattern across existing systems: research and commercial tools tend to address question generation in isolation, optimising for fluency or accuracy while neglecting the broader assessment workflow. Specifically, four gaps remain unaddressed in the existing literature.

First, most systems generate questions without retrieval grounding. They rely on the LLM's parametric knowledge rather than the specific lecture materials uploaded by a lecturer, making the output unverifiable and potentially irrelevant to the course content.

Second, exam-pattern conditioning is largely absent. No reviewed system allows a lecturer to supply sample exam questions and use them to condition the style, structure, and difficulty distribution of the generated output.

Third, Bloom's taxonomy control is not integrated into the generation pipeline. While educational theory recommends distributing assessment questions across cognitive levels, existing systems do not enforce or evaluate this distribution.

Fourth, no reviewed system integrates question generation with a student practice

workflow. The generated questions are treated as a one-time output rather than as content for an interactive learning environment with attempt tracking, score history, and confidence trend visualisation.

QuizGen addresses all four gaps within a single unified system. It combines semantic retrieval from uploaded lecture materials, exam-pattern conditioning via few-shot prompt injection, explicit Bloom-level and difficulty control, a grounding evaluation component, and a web-based quiz practice interface with per-quiz performance analytics. This combination distinguishes QuizGen from all systems reviewed in this chapter and positions it as a more complete solution for lecture-grounded, pattern-aware MCQ generation and practice.

CHAPTER III

METHODOLOGY AND SYSTEM DESIGN

3.1 Proposed Framework Overview

QuizGen is designed as a nine-stage pipeline that transforms raw lecture materials into playable, grounded multiple-choice quizzes with performance analytics. Figure 2 illustrates the complete workflow.

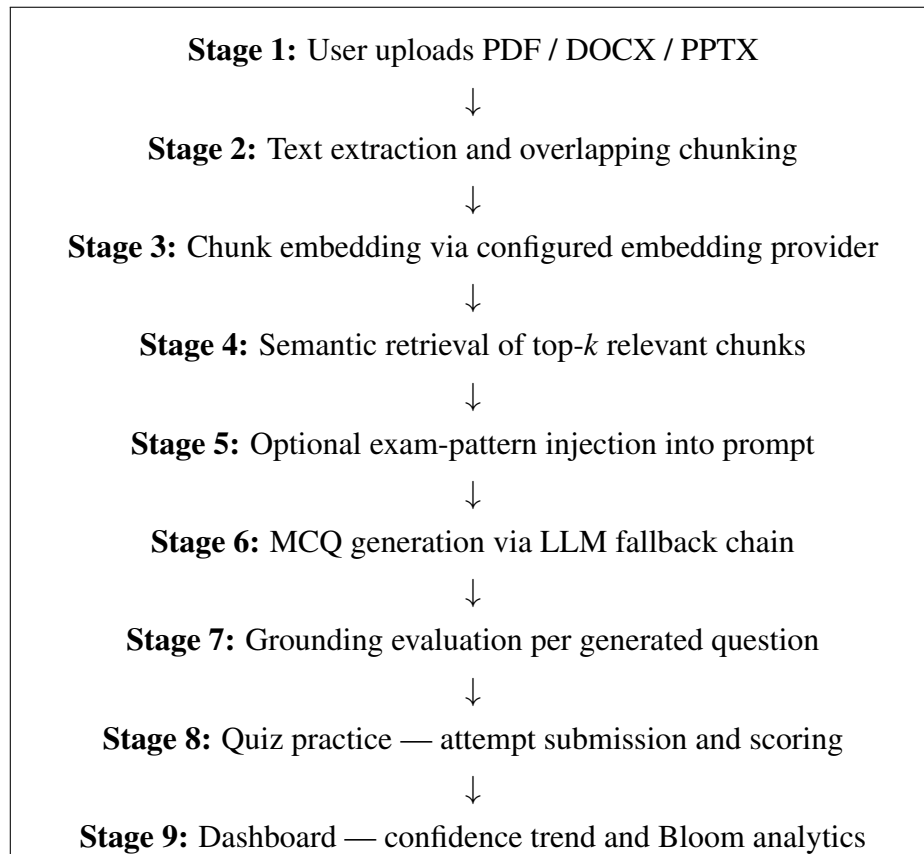


Figure 2: Overall QuizGen pipeline from document upload to analytics.

QuizGen runs as a two-tier web application. The backend is built with FastAPI (Python) and exposes a RESTful API secured with JWT authentication. On the frontend, Next.js and React provide a guided four-step wizard that walks a lecturer from document selection through pattern configuration, generation, and review. SQLite handles persistence. This keeps the system self-contained and suitable for thesis-scale local deployment.

The LLM layer uses a configurable fallback chain. The primary provider is Groq serving the *llama-3.3-70b-versatile* model; if the primary provider is rate-limited, the system automatically retries with alternative Groq models, OpenRouter auto-free models, and finally a local Ollama model. The embedding path uses *gemini-embedding-001* by default and can fall back to the configured OpenRouter embedding model when Gemini embedding calls are unavailable.

3.2 Design Requirements and Architectural Decisions

The architecture of QuizGen was shaped by five design requirements. First, the system had to be usable as an end-to-end thesis prototype, meaning that all major functions should be available through a web interface rather than only through scripts. Second, the system had to keep generated questions traceable to uploaded lecture materials. Third, the generation pipeline had to support exam-pattern conditioning without requiring model fine-tuning. Fourth, evaluation outputs had to be reproducible enough to support thesis claims. Fifth, the implementation had to remain small enough to run on a student development machine.

I made several architectural decisions in response to these requirements. For the backend I chose a modular FastAPI application. It provides clear API boundaries, automatic OpenAPI documentation, and dependency injection for authentication, while

integrating well with Python-based document processing libraries. On the frontend, Next.js supports a responsive route-based workflow with separate pages for quiz creation, practice, attempt review, history, dashboards, settings, and batch generation.

SQLite was selected for persistence because it is sufficient for the single-user and small-classroom scale of the thesis prototype. The design does not depend on SQLite-specific features; the main entities can be moved to PostgreSQL in future work. Similarly, embeddings are stored in the relational database rather than in a separate vector database to keep the deployment simple. This is acceptable for the thesis dataset, where the number of chunks is small enough for in-process cosine-similarity search.

The LLM provider layer was separated from the generation logic so that provider failures do not require changes to prompts or API endpoints. The `llm_router` service receives a normalized request, selects a provider, records usage metadata, and returns a normalized response. This separation allows the rest of the system to treat Groq, Gemini, OpenRouter, and Ollama as interchangeable generation backends. It also supports the usage dashboard by logging latency, status, and token consumption for every provider call.

3.3 Document Processing and Chunking

When a user uploads a lecture file, the backend extracts its raw text content and splits it into overlapping word-level chunks. Three file formats are supported: PDF, DOCX, and PPTX. Each format is handled by a dedicated parser in the document processor service, which normalises the extracted text before chunking.

Chunking is necessary because LLM context windows impose token limits, and embedding quality degrades when inputs are too long. The system applies a fixed-size chunking strategy with a configurable overlap. Table 2 summarises the document

processing configuration used in this thesis.

Table 2: Document processing configuration.

Parameter	Value
Supported formats	PDF, DOCX, PPTX
Chunk size	2,000 words
Chunk overlap	200 words
Embedding provider	Gemini by default, OpenRouter fallback
Retrieval top- <i>k</i>	3 chunks (evaluation setting)
Maximum upload size	50 MB

The overlap of 200 words ensures that concepts spanning a chunk boundary are not severed. When an explanation begins near the end of one chunk and continues into the next, both chunks independently retain enough context for the embedding model to capture the full semantic meaning. This is particularly important for definitions and multi-step explanations that are common in lecture slides.

3.4 Embedding and Retrieval

After chunking, each text segment is converted into a dense vector representation. The default embedding path uses *gemini-embedding-001*; if Gemini embedding calls are unavailable and an OpenRouter key is configured, the router can fall back to the configured OpenRouter embedding model. These vectors are stored in the `chunk_embeddings` table alongside the chunk text and a reference to the parent document. Because embeddings are precomputed at upload time, retrieval at generation time requires no additional embedding API calls for the source side.

The Gemini embedding documentation states that *gemini-embedding-001* returns a 3072-dimensional embedding by default and supports configurable output dimensionality from 128 to 3072, with 768, 1536, and 3072 recommended as common output sizes

[35]. When the Gemini embedding path is used, the implemented backend does not request a smaller output dimensionality in `llm_router.py`; therefore the stored vectors use the provider default. Within a single run, the same configured embedding provider is used for document chunks, retrieval queries, and semantic grounding, which keeps vector spaces consistent across the pipeline.

When a lecturer initiates a generation request, the system constructs a retrieval query from available pattern signals (sample questions, optional pattern language, and requested difficulty distribution), then falls back to document-leading text if these signals are unavailable. The query is embedded using the same model. The cosine similarity between the query embedding and each stored chunk embedding is computed, and the top- k chunks with the highest similarity scores are selected as the source context. This process is illustrated in Figure 3.

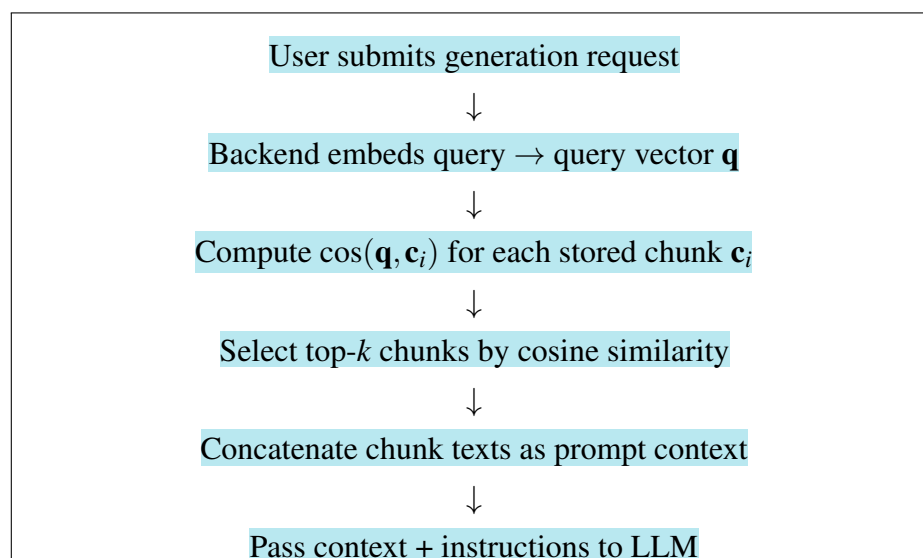


Figure 3: RAG retrieval sequence from query to prompt context.

The selected chunks are joined with separator tokens and inserted into the generation prompt as the *Source Material* section. This ensures that every generated question

has access to relevant lecture content at generation time, directly reducing the risk of hallucinated or unsupported questions.

Cosine similarity is used rather than Euclidean distance because the retrieval task is primarily about semantic direction in the embedding space. The implementation in `embedder.py` explicitly normalizes the query vector and stored chunk vectors before computing the dot product. This makes the score less sensitive to vector magnitude and more focused on whether the query and chunk point toward similar meanings. A plain dot product would mix semantic direction with vector length, while Euclidean distance would treat absolute position and scale as more important than the angular relationship that embedding-search systems commonly use for semantic similarity.

The default retrieval path proceeds in two stages. In the first stage, `cosine_search` returns the top- k chunks by cosine similarity; the core evaluation fixed $k = 3$ for the RAG-only and full-system baselines, while the production default is $k = 8$. In the second stage, `select_relevant_chunks` applies an independent ceiling of `max_chunks=8` on how many of those retrieved chunks are forwarded to the generator. In the evaluation, $k = 3$ meant that at most three chunks were ever retrieved, so the `max_chunks` ceiling never became the binding constraint; the ceiling matters for future experiments that raise k beyond 8. Both parameters are configurable through the service interface and the stored configuration snapshot without changing the database schema.

After the advisor review, I extended the evaluation runner with two optional retrieval modes for ablation analysis. The *hybrid* mode combines dense cosine retrieval with BM25 lexical retrieval [17]. Dense retrieval helps when the query and source use different wording for the same concept, while BM25 helps preserve exact keywords, abbreviations, and positive/negative qualifiers. The two ranked lists are combined using

Reciprocal Rank Fusion (RRF), which assigns a larger score to chunks that appear near the top of either ranking [19]. The *hybrid-rerank* mode then sends the post-fusion candidate chunks to an LLM reranker, following the same general motivation as neural passage reranking work [20]. This reranking path is reported in Chapter V as an ablation, not as the default production retrieval mode, because the thesis-scale results show useful signals but not a clean improvement across all metrics.

3.5 Exam Pattern Extraction

QuizGen allows a lecturer to create a named *exam pattern* by providing sample exam questions and configuration parameters. The pattern is stored as a JSON document in the `patterns` table and captures three types of information: the question type distribution (for example, 70% MCQ and 30% short answer), the average question length in words, and a set of up to five sample questions that serve as few-shot demonstrations of the desired exam style.

When a pattern is selected during generation, the system injects its configuration and sample questions directly into the generation prompt. The sample questions are presented under the heading *Example Questions (match this style and format)*, instructing the LLM to reproduce the same phrasing patterns, option structure, and difficulty feel as the provided examples. This conditioning mechanism requires no model fine-tuning and is therefore compatible with any API-based LLM provider.

Figure 4 shows the pattern configuration step in the QuizGen web interface.

QuizGen + Create new quiz

Generate a new quiz

Upload a lecture, optionally pin an exam pattern, and produce grounded MCQs aligned to Bloom's taxonomy.

Workflow: 1. Source (Pick lecture material) | 2. **Pattern** (Tune exam style) | 3. Generate (Confirm settings) | 4. Review (Inspect & export)

Pattern & difficulty
Optionally pin to an extracted exam pattern and shape Bloom-level distribution.

Exam pattern Refresh list

- No pattern
- No pattern
- ISM Final

Difficulty distribution

40% EASY 40% MEDIUM 20% HARD

Drag the edges between colors (or use arrow keys when a handle is focused). Total stays 100%.

Flow Summary:

Source	5
Pattern	None
Questions	10 MCQs
Language	English
Difficulty	40 / 40 / 20

Step 2 of 4 · Each step gates the next. Settings persist if you go back.

← Back Continue →

Figure 4: Exam pattern and difficulty configuration step in the QuizGen wizard.

3.6 MCQ Generation Method

The generation prompt is assembled from four components: (1) the retrieved source chunks as the *Source Material*, (2) generation instructions specifying the number of questions, question types, and target language, (3) an optional difficulty distribution specifying the percentage of easy, medium, and hard questions, and (4) the optional exam-pattern block containing sample questions and style metadata.

The assembled prompt is sent to the primary LLM provider, which is Groq serving the *llama-3.3-70b-versatile* model. The system instructs the model to return a valid JSON array and does not allow markdown code fences or free-text preambles in the response. Each element of the array corresponds to one generated question and must conform to the schema described in Table 3.

The actual prompt template used by the backend is stored in `backend/app/prompts/v1/quest.`

A shortened excerpt is shown below, preserving the implemented placeholders:

Source Material

{retrieved_chunks_joined_with_separators}

Generation Instructions

- Generate exactly {num_questions} questions
- Question types to include: {question_types}
- Language: {language}
- Prompt version: v1

Difficulty Distribution (MUST follow exactly)

- {level}: {percentage}% (~{count} questions)

Pattern Requirements

- Question type distribution: {pattern_config.question_types}
- Average question length: ~{pattern_config.avg_length} words

Example Questions (match this style and format):

Example 1:

{sample_question}

Output Format

Return a JSON array of objects with question, options, answer, explanation, difficulty, and bloom_level.

The generation provider is called through llm_router.py. The Gemini and

Groq request payloads set temperature to 0.7, and the Groq path also passes the same value through its request options. A temperature of 0.7 allows variation in stems and distractors, which is useful for quiz generation, while remaining constrained by the source material, schema, and difficulty distribution. The project does not currently set a fixed random seed, so reproducibility is handled by storing the prompt version, provider, token usage, and configuration snapshot with each generation.

Table 3: MCQ output schema.

Field	Type	Description
type	string	Question type: mcq, true_false, fill_blank
question	string	The question stem
options	array	Four labelled options (A–D) for MCQ; empty otherwise
answer	string	The correct answer
explanation	string	Brief explanation referencing the source material
difficulty	string	easy, medium, or hard
bloom_level	string	One of six Bloom levels (see Section 2.4)

The Bloom level field is populated by the LLM based on the cognitive demand of the question. The six valid values are remember, understand, apply, analyze, evaluate, and create, corresponding to the revised Bloom’s taxonomy [11]. When a difficulty distribution is specified in the prompt, the model is instructed to generate the target number of questions at each difficulty level as closely as possible.

3.7 Prompt Assembly and Output Validation

The prompt assembly stage is responsible for converting user configuration and retrieved source material into a controlled instruction for the LLM. The backend builds

the prompt in a fixed order: role instruction, source material, optional exam-pattern examples, generation constraints, output schema, and validation rules. Keeping this order stable is important for reproducibility because even small prompt rearrangements can change the model's behaviour.

The role instruction frames the model as a university lecturer who must generate assessment questions only from the provided source material. The source section contains the retrieved chunks and their internal chunk identifiers. The optional pattern section lists the sample questions saved by the lecturer. The constraint section specifies the number of questions, language, difficulty distribution, and Bloom-level targets. The schema section instructs the model to return only JSON so that the response can be parsed automatically.

After the model returns a response, the `question_generator` service parses and normalizes the output. The parser removes accidental markdown code fences, tries direct JSON parsing, can extract the first complete JSON array from surrounding text, removes common trailing-comma and JavaScript comment issues, and can salvage a truncated JSON array up to the last complete object. If parsing still fails, the system asks the provider for a JSON-only repair response. If the retry also fails, generation is marked as failed and the error is surfaced to the frontend.

The current implementation should be described as tolerant normalization, not strict validation. For each dictionary-like item, the backend assigns an incremental id, forces the type to `mcq`, truncates options to the first four entries, preserves the answer and explanation fields if present, defaults missing difficulty to `medium`, and maps invalid Bloom labels from the difficulty label. The scoring route later canonicalizes answer letters and option text when a student submits an attempt. Therefore, the prompt requests a complete MCQ schema, but the parser does not reject every incomplete item.

Table 4 summarizes the major parsing and normalization behaviours applied before a generated question is stored.

Table 4: Prompt output parsing and normalization behaviour.

Parser behaviour	Purpose
Response must parse as JSON	Prevents storing free-text LLM responses
Dict/list wrappers are normalized	Handles common provider response shapes
Non-dictionary items are skipped	Prevents unusable primitive values from being stored
Options are truncated to four entries	Matches the MCQ interface expectation
Missing difficulty defaults to medium	Supports downstream grouping
Invalid Bloom labels are remapped	Keeps Bloom labels inside allowed values
Answer text is preserved for quiz scoring	Allows later answer canonicalization

This distinction matters for thesis accuracy. The implementation enforces a parseable JSON structure and normalizes expected fields, but it relies on the prompt and human review to ensure that every generated option set, answer, and explanation is pedagogically strong.

3.8 Grounding and Hallucination Proxy

After generation, each MCQ is scored by the grounding evaluator, which estimates the degree to which the question text, correct answer, and explanation are supported by the source chunks. The evaluator concatenates these three fields into a single passage and computes its cosine similarity against the source document embedding. Because raw cosine similarity lies in $[-1, 1]$, it is normalised to $[0, 1]$ using the transformation $s = (\cos + 1)/2$ before applying the grounding thresholds.

More formally, for a generated question q_i , the evaluator builds:

$$p_i = \text{question}_i \parallel \text{answer}_i \parallel \text{explanation}_i$$

where $\parallel$ denotes string concatenation. It then embeds p_i and the source document text into vectors $\mathbf{p}_i$ and $\mathbf{d}$. The raw semantic score is:

$$\cos(\mathbf{p}_i, \mathbf{d}) = \frac{\mathbf{p}_i \cdot \mathbf{d}}{\|\mathbf{p}_i\| \|\mathbf{d}\| + 10^{-12}}$$

and the reported grounding score is:

$$s_i = \max(0, \min(1, (\cos(\mathbf{p}_i, \mathbf{d}) + 1)/2)).$$

For example, if a generated DBMS question and its explanation produce a raw cosine similarity of 0.46 against the uploaded database lecture text, the normalized grounding score is $(0.46 + 1)/2 = 0.73$, which falls into the `well_grounded` range. If a different question produces a raw cosine similarity of -0.10, the normalized score is 0.45 and the item is flagged as `poorly_grounded`. These scores are not proof of factual correctness; they are triage signals that help reviewers focus on suspicious items.

Table 5 lists the three grounding status levels and their corresponding score ranges.

Table 5: Grounding status levels and semantic score thresholds.

Status	Score range	Interpretation
<code>well_grounded</code>	$s > 0.70$	Strong semantic overlap with source
<code>partially_grounded</code>	$0.45 < s \leq 0.70$	Moderate overlap; review recommended
<code>poorly_grounded</code>	$s \leq 0.45$	Weak overlap; likely unsupported

In addition to the semantic score, the evaluator computes a keyword overlap baseline by comparing the set of content tokens (words of three or more characters) in the generated passage against the source document tokens. This secondary metric provides an interpretable count of matched terms and helps the lecturer identify which specific source words were covered. The grounding score is reported as a review aid; it highlights questions that may require additional validation but does not guarantee factual correctness for well-grounded questions.

3.9 Quiz Practice and Confidence Trend

Once a generation is reviewed and saved, it becomes a named quiz that any authenticated user can attempt. The quiz practice interface presents one question at a time, records the user's selected option, and tracks elapsed time. On submission, the backend stores the attempt in the `quiz_attempts` table, recording the generation identifier, the user's answers, the final score, the total time taken, and per-question Bloom-level correctness.

The confidence trend is computed as a user-level chronological sequence of attempt score percentages ordered by submission time. The current dashboard implementation does not group the series by quiz; instead, it visualises all stored attempts for the authenticated user in one timeline. This longitudinal view still indicates whether overall practice performance is improving, stable, or declining.

Figure 5 shows the practice and confidence trend workflow.

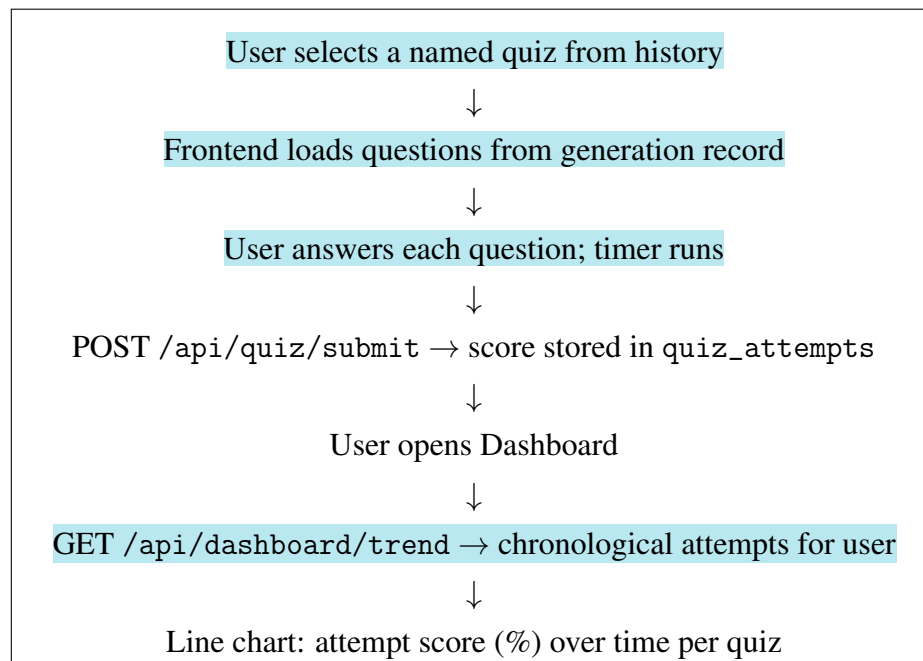


Figure 5: Quiz practice and confidence trend sequence.

3.10 Database Design

A single SQLite database holds all persistent state across eight core tables.

Table 6 lists each entity and its purpose.

Table 6: Database entities and their roles.

Entity	Role
users	Account identity, hashed password, and role
documents	Uploaded files, raw text, and processed chunks
chunk_embeddings	Chunk vectors for semantic retrieval
patterns	Exam templates, sample questions, style config
generations	Generated quiz records with prompt and config snapshot
quiz_attempts	User answers, scores, timing, and Bloom accuracy
batch_jobs	Batch generation status, progress, and per-document results
api_calls	Provider usage, token count, and latency logs

The key relationships are as follows. A user owns many documents and patterns. A generation references one document (as source) and optionally one pattern (as style guide). A quiz_attempt references one generation and one user. A batch_job references a user and stores the list of document identifiers processed as independent generations. The api_calls table records every outbound LLM and embedding call for usage tracking and provider-level diagnostics.

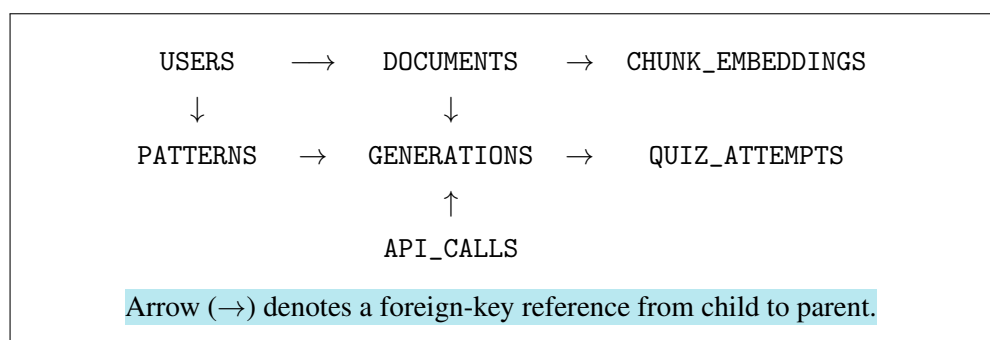


Figure 6: Simplified entity–relationship overview of the QuizGen database.

The generations table stores a config_snapshot JSON field that captures the exact chunk size, embedding model, prompt version, and provider used for each run. This reproducibility artifact ensures that evaluation results reported in Chapter V can be traced back to a specific system configuration.

3.11 Security and Reproducibility Considerations

Although QuizGen is a thesis prototype, the system includes several security controls because the workflow handles uploaded documents, generated assessment content, and student attempt records. User accounts are protected by hashed passwords, and authenticated API requests use JWT bearer tokens. Protected endpoints resolve the current user through a backend dependency instead of trusting a user identifier supplied

by the client. This design prevents a user from reading or modifying another user's documents, patterns, generations, or quiz attempts through ordinary API calls.

Uploaded files are processed only after the backend checks their extension and size. Unsupported formats are rejected, and the maximum upload size is bounded by configuration. Secrets such as API keys and JWT signing keys are loaded from environment variables, not hard-coded in source files. These controls are basic, but they are important for a system that may eventually handle private lecture material.

Reproducibility is handled at two levels. At the application level, each generation record stores the prompt version, provider, model, source document, optional pattern, selected difficulty settings, and generated question payload. At the evaluation level, the experiment runner stores CSV outputs for each run, aggregated comparison tables, and failure-analysis reports. The post-review evaluation runner also writes checkpoint records after completed cases, so a long run can be inspected even if quota limits or process interruption stop the experiment before all baselines finish. Together, these records allow the thesis results to be checked against the concrete configuration that produced them.

The main reproducibility limitation is the use of external LLM APIs. Even with fixed prompts and a fixed random seed in the evaluation runner, provider-side model updates can change outputs over time. For this reason, the thesis reports the cached evaluation run used for the final comparison and treats future reruns as additional evidence rather than as replacements for the documented result set. The implementation also supports provider fallback through OpenRouter auto-free models and a local Ollama route. These fallbacks improve demonstration resilience under quota pressure, but they are treated as engineering support rather than as a primary research contribution.

CHAPTER IV

IMPLEMENTATION AND RESULTS

4.1 Development Environment

QuizGen is implemented as a two-tier web application composed of a Python backend and a Next.js frontend. Table 7 lists the full technology stack used in this thesis implementation.

Table 7: QuizGen technology stack.

Layer	Technology	Notes
Backend language	Python 3.10+	
Backend frame- work	FastAPI	RESTful API, JWT authentication
ASGI server	Uvicorn	Hot-reload in development
Database	SQLite	Single-file, zero-configuration
Frontend frame- work	Next.js + React	App Router, TypeScript
UI components	shadcn/ui + Tailwind CSS	
Charts	Recharts + custom SVG	Evaluation trends, confidence trend, Bloom breakdown
LLM primary	Groq API	<i>llama-3.3-70b-versatile</i>
LLM fallbacks	Groq API (multiple)	Four alternative Groq models
LLM cloud fallback	Gemini + OpenRouter	Gemini legacy path and OpenRouter auto-free models
LLM local fallback	Ollama	<i>gemma4:e4b-it-q4_K_M</i> , offline
Embeddings	Gemini + OpenRouter	<i>gemini-embedding-001</i> ; OpenRouter fallback model
Local run	<code>start-all.bat</code>	Starts both services on Windows
Containerization	Docker Compose	Two-service stack with named volume

The backend and frontend can be started locally on Windows using the provided

start-all.bat script, which checks for Python and Node.js, installs dependencies if needed, and launches both services in separate terminal windows. The backend runs on port 8000 and the frontend on port 3000. For containerised deployment, a docker-compose.yml file is provided that builds both services from their respective Dockerfiles and mounts a named volume (quizgen-data) for SQLite database persistence. An interactive API reference is available at <http://localhost:8000/docs> via FastAPI's built-in Swagger UI.

4.2 Project Structure and Configuration

The implementation is organised into separate backend, frontend, evaluation, and documentation directories. This structure keeps the production application code separate from thesis evidence and experiment artifacts. Table 8 summarizes the major directories and their roles.

Table 8: Repository structure used by the QuizGen implementation.

Path	Responsibility
backend/app/api	FastAPI route modules for user-facing endpoints
backend/app/services	Document processing, retrieval, generation, evaluation logic
backend/app/models	Database model definitions and persistence objects
backend/app/schemas	Request and response schemas used by the API layer
frontend/src/app	Next.js pages and route-level application layout
frontend/src/components	Reusable workflow and UI components
frontend/src/features	Feature-specific views such as dashboard and evaluation
eval	Reproducible evaluation runner, dataset, and result outputs
doc/thesis	Thesis manuscript, evaluation notes, and documentation artifacts

Configuration is controlled primarily through environment variables. The backend reads values such as DATABASE_PATH, JWT_SECRET, GEMINI_API_KEY,

GROQ_API_KEY, MAX_UPLOAD_SIZE_MB, and LLM_FALLBACK_CHAIN. This keeps deployment-specific settings outside the codebase and allows the same application to run with dummy keys during UI smoke tests or with real keys during a final generation demo.

The frontend centralizes API calls in frontend/src/lib/api.ts. This file attaches the bearer token to protected requests and normalizes API errors before they are shown in the interface. Keeping API access in one module reduces duplication across the workflow wizard, dashboard, quiz pages, and evaluation view.

4.3 Backend Implementation

The backend is organised into two layers: API routers that handle HTTP requests and services that encapsulate business logic. Table 9 lists all modules and their responsibilities.

Table 9: Backend API modules and services.

Type	Module	Responsibility
Router	auth	Registration, login, JWT token issuance
Router	documents	Upload, text extraction, chunk storage
Router	patterns	Create, list, and delete exam patterns
Router	generations	Trigger MCQ generation, retrieve results
Router	batch	Batch jobs that create independent quizzes per document
Router	quiz	Submit attempts, retrieve attempt records
Router	dashboard	Confidence trend and Bloom breakdown data
Router	usage	API call logs and provider status
Router	eval	Serve latest and historical evaluation artifacts to admin dashboard
Router	settings	Return live model, fallback, and RAG configuration
Service	document_processor	PDF/DOCX/PPTX extraction and chunking
Service	embedder	Batch embedding through the configured embedding router
Service	chunk_selector	Cosine-similarity retrieval of top- <i>k</i> chunks
Service	pattern_analyzer	Parse and validate exam-pattern JSON
Service	question_generator	Prompt assembly, LLM call, and JSON parsing
Service	question_extractor	Extract sample questions from uploaded patterns
Service	accuracy_evaluator	Grounding score computation
Service	llm_router	Provider fallback chain and API call logging
Service	batch_processor	Placeholder for future extraction of batch orchestration logic
Service	rate_limit	Per-user request rate limiting

The `main.py` entry point registers ten routers under the `/api` prefix, configures CORS to allow the frontend origin, and initialises the SQLite schema on startup via `init_db()`. JWT authentication is enforced at the router level using FastAPI dependency injection; protected endpoints receive a verified user object rather than a raw token.

The `llm_router` service implements a configurable fallback chain read from the `LLM_FALLBACK_CHAIN` environment variable. The default chain begins with Groq serving *llama-3.3-70b-versatile*, then cascades through four additional Groq models,

OpenRouter auto-free models, and finally Ollama with a local *gemma4:e4b-it-q4_K_M* model as an offline last resort. Every outbound LLM and embedding call is logged to the `api_calls` table with provider name, model, latency, token count, and status, enabling per-provider usage diagnostics in the usage dashboard.

4.4 API Workflow Implementation

The API workflow follows the same sequence as the user-facing wizard. A user first authenticates through the auth router, uploads or selects source material through the documents router, optionally creates an exam pattern through the patterns router, and then submits a generation request to the generations router. The generated quiz can later be opened through the history view or attempted through the quiz router.

Table 10 lists representative endpoints used in the main workflow.

Table 10: Representative API endpoints in the QuizGen workflow.

Endpoint	Method	Purpose
<code>/api/auth/register</code>	POST	Create a new user account
<code>/api/auth/login</code>	POST	Return a JWT access token
<code>/api/documents/upload</code>	POST	Upload and process lecture material
<code>/api/documents/</code>	GET	List documents owned by the current user
<code>/api/patterns/</code>	POST	Create an exam-pattern record
<code>/api/generations/</code>	POST	Generate a question set from source material
<code>/api/generations/</code>	GET	List previous generations and titles
<code>/api/quiz/submit</code>	POST	Submit answers for a generated quiz
<code>/api/dashboard/trend</code>	GET	Return confidence trend data
<code>/api/usage/</code>	GET	Return provider usage statistics

The generation endpoint coordinates the most complex backend operation. It validates that the source document belongs to the current user, retrieves the selected pattern if provided, embeds the generation query, selects the top relevant chunks, builds

the prompt, calls the LLM router, validates the returned JSON, computes grounding information, and stores the final generation record. Each step is isolated in a service module so that retrieval, prompt assembly, and scoring can be tested or replaced independently.

The API also includes validation guards for common failure cases. Missing documents or patterns return 404. Invalid generation titles return 400. Requests without a bearer token return 401. Normal users are denied access to admin evaluation endpoints with 403. These response codes are important because the frontend uses them to show clear user-facing messages rather than generic application failures.

4.5 Frontend Implementation

The frontend is built with Next.js using the App Router and TypeScript. The current user-facing interface uses a route-based Atelier layout rather than the earlier tabbed workflow. A shared `AtelierShell` provides the sidebar, breadcrumb, theme control, and recent-generation navigation, while individual App Router pages implement the major tasks: `/workflow` for quiz creation, `/history` for saved generations and attempts, `/quiz/[genId]` for practice, `/quiz/attempt/[attemptId]` for review, and separate dashboard, usage, evaluation, settings, and batch routes. The main quiz creation flow remains a guided four-step wizard: Source, Pattern, Generate, and Review. The wizard enforces step ordering so that generation cannot proceed without a source document.

Figure 7 shows the Source step, where the user uploads a lecture file or selects a previously processed document.

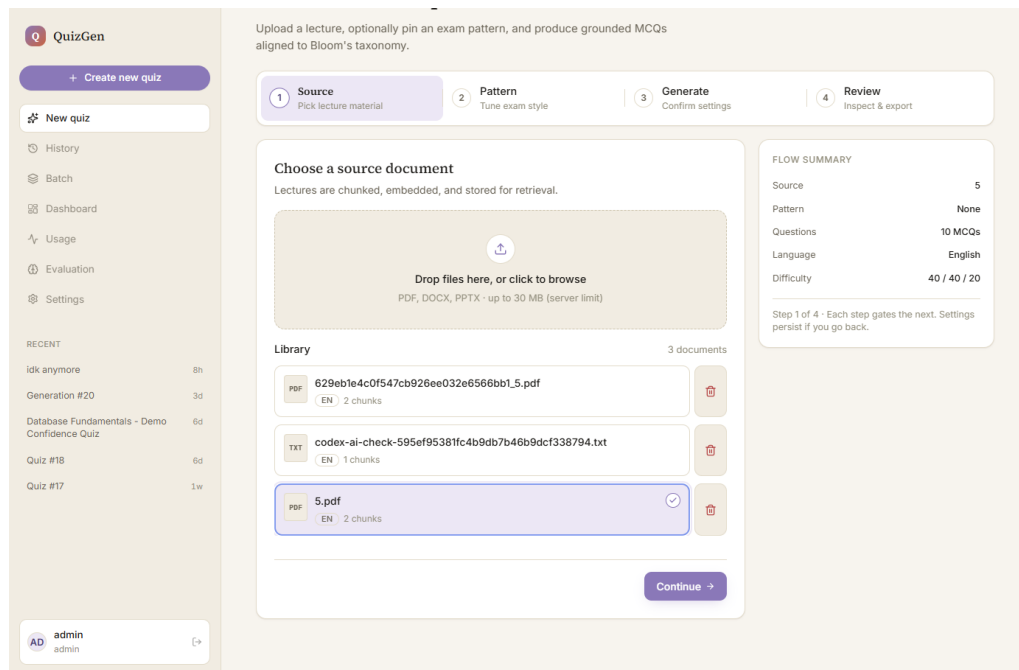


Figure 7: Source step: document upload and selection in the QuizGen wizard.

The Pattern step, shown previously in Figure 4, allows the user to attach an optional exam pattern, choose the output language, and configure the difficulty distribution using a compact slider control. If no pattern is selected, the system generates questions using the retrieved source chunks alone, without style conditioning.

Figure 8 shows the Generate confirmation step, where the user reviews the selected source, optional pattern, question count, language, and difficulty distribution before submitting the generation request to the backend.

QuizGen

+ Create new quiz

New quiz

History

Batch

Dashboard

Usage

Evaluation

Settings

RECENT

idx anymore 8h

Generation #20 3d

Database Fundamentals - Demo Confidence Quiz 6d

Quiz #18 6d

Quiz #17 1w

admin admin

WORKFLOW

Generate a new quiz

Upload a lecture, optionally pin an exam pattern, and produce grounded MCQs aligned to Bloom's taxonomy.

1 Source Pick lecture material

2 Pattern Tune exam style

3 **Generate** Confirm settings

4 Review Inspect & export

Confirm & generate

Review the setup. The model retrieves grounded chunks, then generates with your pattern.

SOURCE
5.pdf
2 chunks - EN

PATTERN
ISM Final
exam style

OUTPUT
10 MCQs - English
40/40/20 split

Ready when you are

Generation usually takes 8-14 seconds depending on document length and provider.

Generate 10 MCQs

← Back

Continue →

FLOW SUMMARY

Source	5
Pattern	ISM Final
Questions	10 MCQs
Language	English
Difficulty	40 / 40 / 20

Step 3 of 4 · Each step gates the next. Settings persist if you go back.

Figure 8: Generate step: quiz title assignment before submitting the generation request.

Figure 9 shows the Review step, where generated questions are displayed with their answer options, difficulty label, Bloom level, and grounding status. The user can rename the saved generation, export the questions, run an optional LLM grounding check, or launch the same generation directly in practice mode.

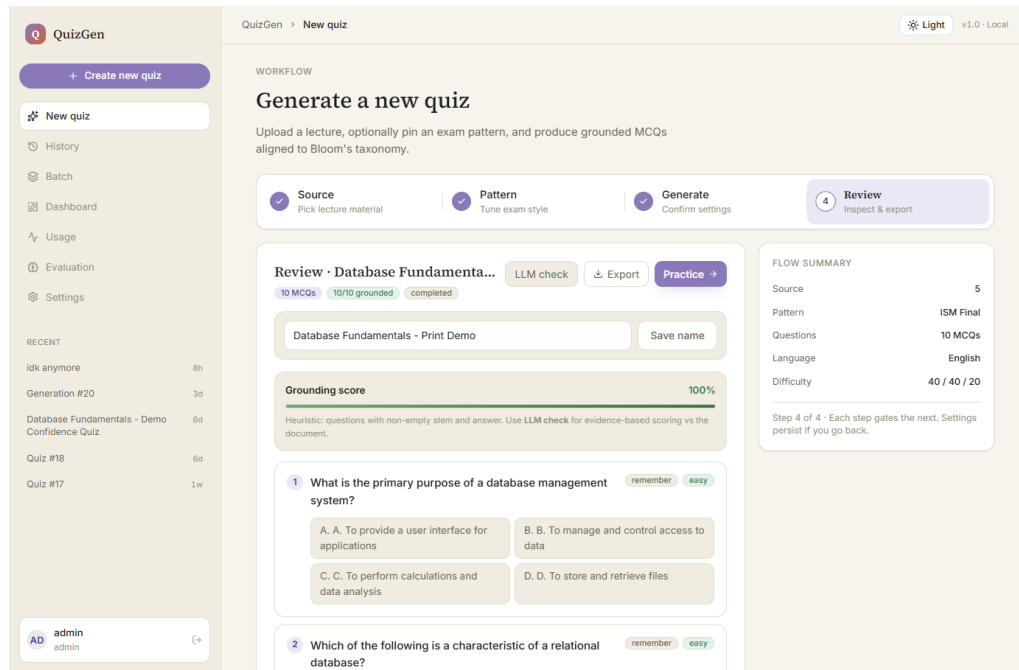


Figure 9: Review step: generated quiz with grounding status, Bloom labels, and difficulty indicators.

Navigation is protected by a JWT-aware auth-provider context, which stores the access token in client storage and attaches it as a Bearer header to all API requests. Unauthenticated users are redirected to the login page. Toast notifications surface API errors and quota warnings without blocking the interface.

4.6 User Interface State and Error Handling

The frontend workflow is stateful because each step depends on choices made in earlier steps. The Source step stores the selected document identifier. The Pattern step stores either a selected pattern identifier or an explicit “no pattern” choice. The Generate step stores the requested question count, language, and difficulty distribution. The Review step stores the generated question payload, editable quiz title, export state, and any grounding results returned by the backend.

Several guards are implemented to prevent invalid requests. The user cannot continue from Source without selecting or uploading a document. The difficulty distribution is checked before generation so that invalid percentage totals do not reach the backend. Question count inputs are bounded to avoid accidental requests for an unreasonable number of questions. Generation titles are limited to a practical length so that they fit in the history list, recent-generation sidebar, and dashboard selectors. Long source filenames are truncated in the source library so that selection indicators and delete controls remain visible.

Error handling is designed to keep the user inside the workflow. If upload fails, the Source step remains visible and displays the error. If generation fails because of provider quota, JSON parsing, or validation failure, the Generate step surfaces the message and allows the user to retry. If a saved quiz cannot be loaded, the quiz page redirects the user back to a safe history or login state depending on the authentication status.

This behaviour is important for thesis demonstration because the live AI part of the system depends on external providers. Prepared screenshots and cached evaluation results can demonstrate the system even if network quota is unavailable, but the interface still needs to fail cleanly during real use. The usage dashboard supports this by showing provider status and fallback events.

4.7 Quiz Practice Implementation

Once a generation is saved, it appears in the generation history and recent sidebar list and can be launched as a timed quiz. The quiz practice page, located at `/quiz/[genId]`, loads the generation record, presents one question at a time, and runs an elapsed-time counter. The user selects an answer for each question and submits the

attempt; the backend stores the submission in the `quiz_attempts` table and returns the final score alongside per-question correctness.

Figure 10 shows the quiz practice interface.

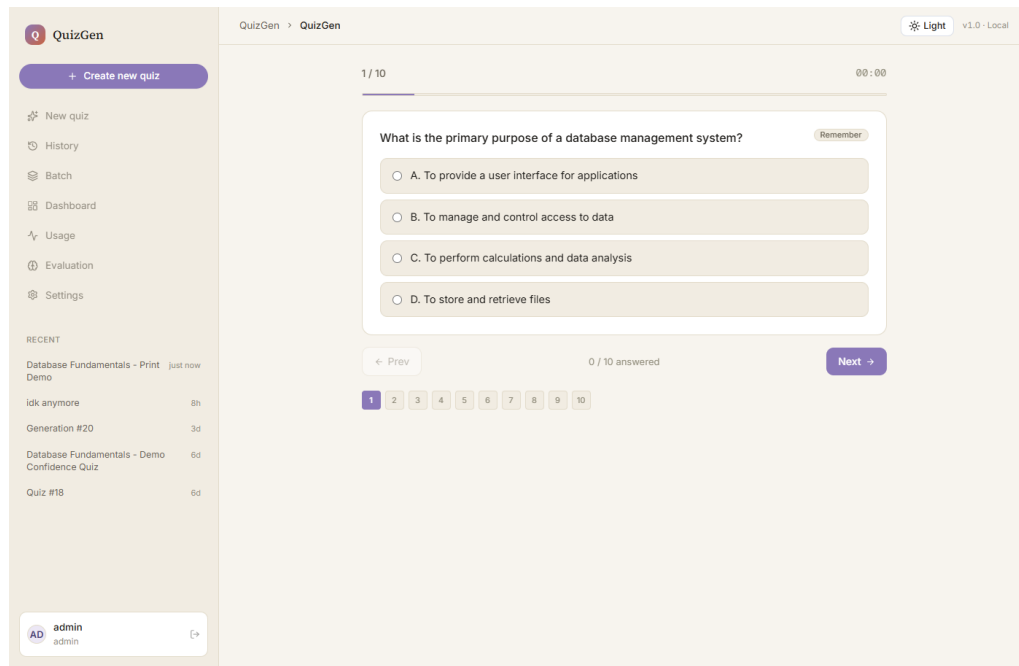


Figure 10: Quiz practice interface showing a question with answer options and the elapsed timer.

After submission, the user is redirected to the attempt review page at `/quiz/attempt/[attemptId]`, which shows each question alongside the user's selected answer, the correct answer, the explanation, and the Bloom level. This view allows the user to identify specific knowledge gaps immediately after each practice session.

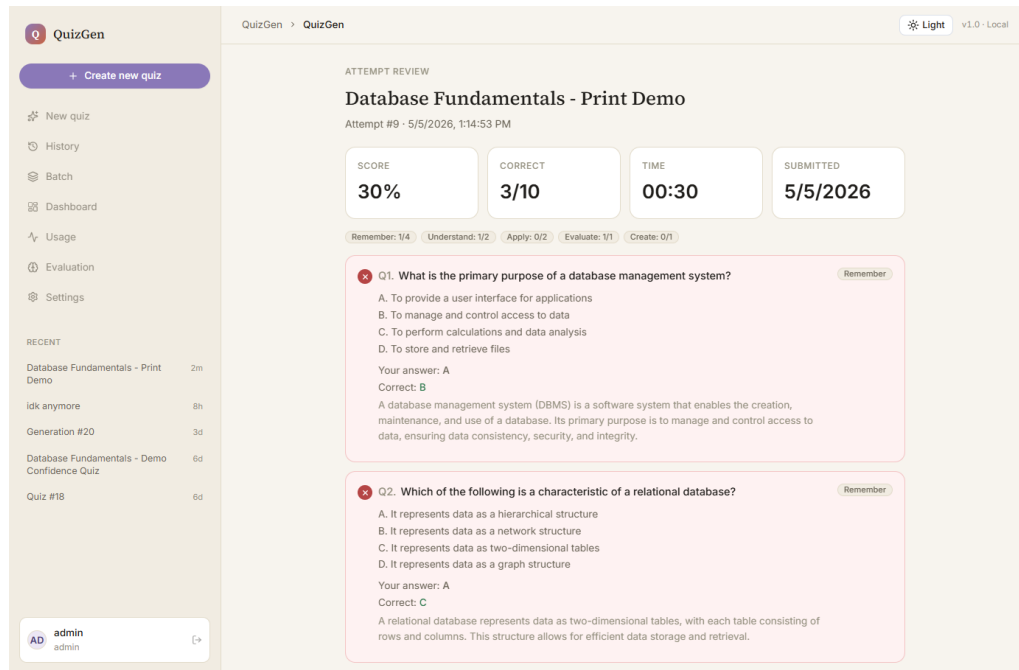


Figure 11: Attempt review page showing per-question correctness, correct answers, explanations, and Bloom levels.

4.8 Dashboard and Usage Implementation

The dashboard frontend is available after login and displays performance data from the dashboard API. In the current backend implementation, the dashboard routes aggregate stored quiz attempts globally rather than filtering by the authenticated user. This limitation is documented in the test run and is treated as a known issue rather than as a production-grade multi-user analytics claim. Its primary visualisation is an SVG confidence trend chart in the Atelier dashboard page. The chart plots attempt score percentage over submission time and is paired with recent attempts and Bloom breakdown summaries.

Figure 12 shows the generation history panel, which includes an inline rename feature so that quizzes can be given meaningful titles after generation.

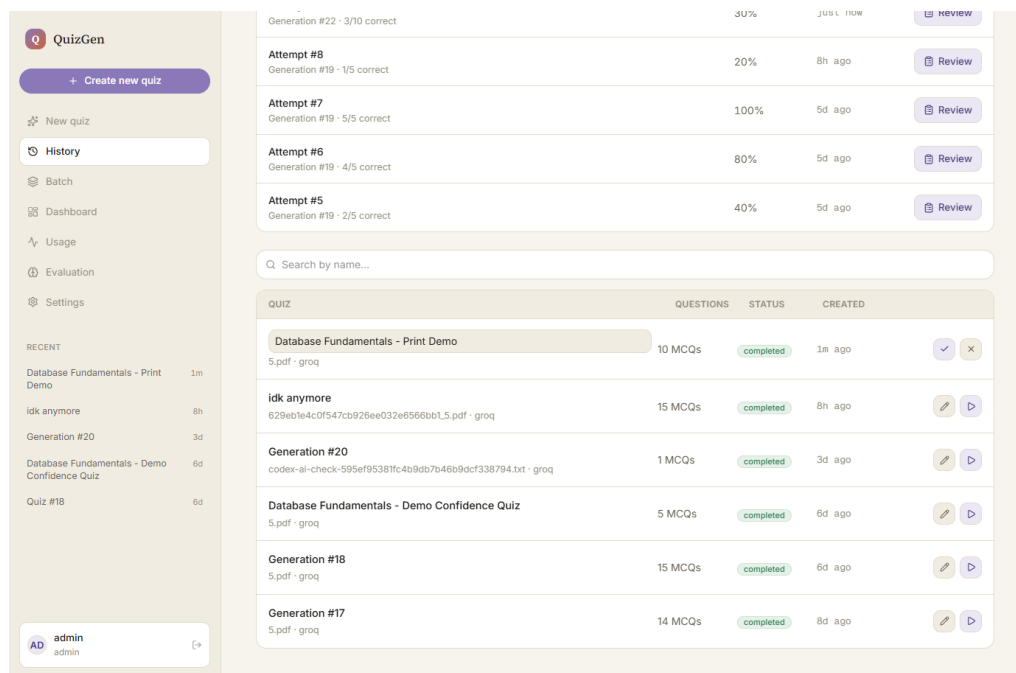


Figure 12: Generation history panel with inline quiz rename functionality.

Figure 13 shows the confidence trend dashboard.

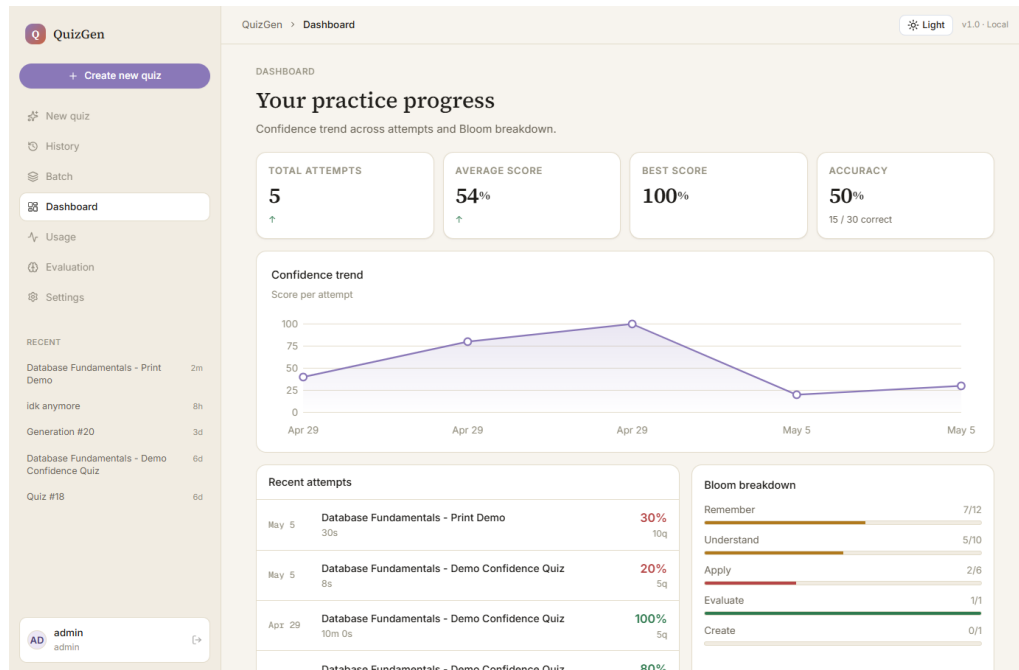


Figure 13: Confidence trend chart showing attempt score percentage over time across stored user attempts.

The usage dashboard, shown in Figure 14, displays provider-level API call statistics, including token consumption, generation count, provider breakdown, recent calls, and fallback-related telemetry. This view allows the user or administrator to monitor which LLM provider is handling the most requests and to identify rate-limiting events.

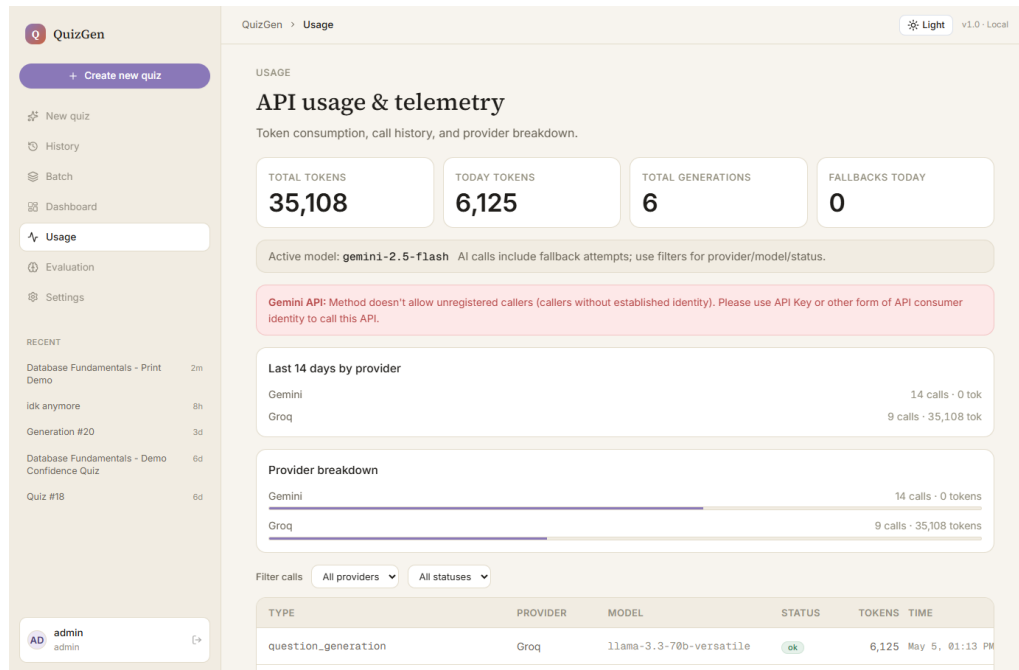


Figure 14: Usage dashboard showing per-provider API call statistics and token consumption.

4.9 Evaluation Dashboard Implementation

The evaluation dashboard provides an in-application view of the system's quantitative performance. It is implemented across four components in the features/evaluation module: `evaluation-dashboard` (the page container), `latest-table` (a summary table of the most recent evaluation run), `history-chart` (a trend chart of metric values across evaluation runs), and `metric-explainer` (inline definitions of each metric for non-expert users). Evaluation data is served by the `/api/eval` router and loaded on demand rather than at startup, so the dashboard does not block the main application during evaluation runs.

Figure 15 shows the evaluation dashboard displaying the latest mean and standard deviation results for the three core baselines.

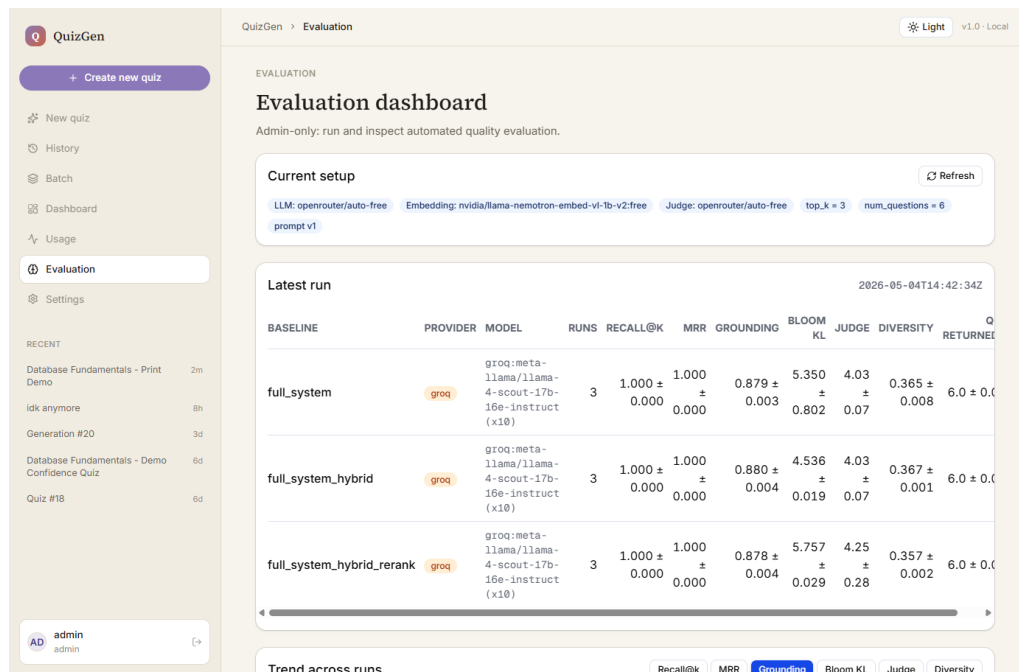


Figure 15: Evaluation dashboard showing the latest baseline comparison results with mean and standard deviation values.

4.10 Testing and Demonstration Readiness

The implementation was checked using a combination of frontend build checks, backend compilation, API smoke tests, and route-render tests. The goal of this testing was not to prove production reliability, but to verify that the main thesis demonstration workflow can be executed and that common failure cases are handled cleanly.

Table 11 summarizes the practical smoke-test evidence recorded for the final implementation. The final visual pass was performed on 5 May 2026 in light mode so that the screenshots remain legible when the thesis is printed.

Table 11: Implementation smoke-test summary.

Area	Result	Evidence
Frontend lint	Pass	npm run lint completed successfully
Frontend production build	Pass	Next.js build passed outside the Windows sandbox
Backend Python compile	Pass	compileall completed for backend modules
Backend health	Pass	/api/health returned {"status": "ok"}
Auth and protected routing	Pass	Admin login, logout, and protected redirect checked
Frontend workflow smoke	Pass	All core pages checked: Source, Pattern, Generate, Review, Quiz, Attempt, History, Dashboard, Usage, Evaluation, Settings, and Batch
Live AI/key smoke	Pass	One demo generation completed and was submitted as a quiz attempt
Full evaluation rerun	Cached	Thesis uses the saved 2026-05-04 evaluation artifacts

The browser smoke test covered authentication, protected route redirect, document selection, long filename display, pattern selection, generation, review rename and export controls, quiz practice, attempt review, History, Dashboard, Usage, Evaluation, Settings, and Batch. The screenshot evidence in Appendix E was captured from this final Atelier frontend pass.

One important issue was identified during testing: the dashboard trend view is not grouped per quiz in the current implementation. Authentication and user filtering are enforced, but the chart aggregates all stored attempts for the authenticated user into one timeline. For the thesis demonstration, this is documented as a product limitation rather than a per-quiz analytics claim.

Overall, the implementation is sufficient to demonstrate the core research workflow: document upload, pattern configuration, MCQ generation, grounding review, quiz practice, attempt review, dashboard visualization, usage monitoring, and evaluation

reporting. The remaining engineering issues are documented as limitations and future work rather than hidden from the evaluation.

CHAPTER V

DISCUSSION AND EVALUATION

5.1 Evaluation Design

The evaluation compares three pipeline configurations on generation quality, retrieval accuracy, and Bloom-level alignment. I used a purpose-built golden dataset containing four documents and ten topics in two languages. Table 12 summarises the evaluation configuration.

Table 12: Evaluation setup.

Parameter	Value
Documents	4 (IT Fundamentals, DB Fundamentals, OS Principles, Network Security)
Topics	10 (7 English, 3 Vietnamese)
Questions per topic	6
Repeats per baseline	3
Total question sets	90 (3 baselines $\times$ 10 topics $\times$ 3 repeats)
Generation model	<i>meta-llama/llama-4-scout-17b-16e-instruct</i> via Groq
Judge model	Core run: <i>gemini-2.5-flash</i> ; post-review ablation: OpenRouter auto-free
Embedding model	Core run: <i>gemini-embedding-001</i> ; post-review ablation: OpenRouter embedding model
Retrieval top- k	3 (RAG only and full system); 1 (vanilla)
Prompt version	v1
Random seed	42

The evaluator is configured through `eval/config.yaml`, but some model resolution details are controlled by runtime environment and router fallback. To keep reproducibility explicit, this thesis treats `runs.csv` and `history.md` as authoritative append-only evidence and treats `comparison.csv`, `details.csv`, and `failure_analysis.md`

as latest-run snapshots.

I compared three core baselines. **Baseline vanilla** uses no retrieval and no pattern: the full raw document text is passed as context with $\text{top-}k = 1$ to approximate a direct LLM call. **Baseline RAG only** applies $\text{top-}k = 3$ semantic retrieval but no exam-pattern conditioning. **Full system** adds exam-pattern conditioning and Bloom-level distribution control on top of the RAG pipeline.

The generation model is fixed to *meta-llama/llama-4-scout-17b-16e-instruct* across all three baselines. This design choice isolates the effect of the pipeline configuration (retrieval and pattern conditioning) from the effect of the model itself, ensuring that differences in the results are attributable to the system components rather than to model variation. The application default model (*llama-3.3-70b-versatile*) is used for production inference but was not used in the controlled evaluation.

After the advisor review, I ran a separate retrieval ablation on the full system only. This ablation keeps the generation pipeline fixed and varies only the retrieval mode: dense retrieval, hybrid dense+BM25 retrieval, and hybrid retrieval followed by LLM reranking. The ablation is reported separately from the three core baselines because it answers a narrower question: whether retrieval reranking reduces topic confusion and improves quality signals within the already pattern-aware pipeline.

5.2 Metrics

Table 13 defines the nine metrics computed by the evaluation pipeline.

Table 13: Evaluation metrics, definitions, and desired direction.

Metric	Symbol	Definition	Better
Recall@ k	$R@k$	Fraction of ground-truth chunks present in the top- k retrieved chunks	Higher
MRR	MRR	Mean reciprocal rank of the first ground-truth chunk in the ranked list	Higher
Semantic grounding	SG	Mean normalised cosine similarity between each question embedding and the source chunk embeddings; range [0, 1]	Higher
Bloom KL divergence	KL	KL divergence between the target Bloom distribution and the observed Bloom label distribution of generated questions; lower means closer alignment	Lower
LLM judge score	J	Mean of four criteria (relevance, correctness, clarity, groundedness) scored 1–5 by a <i>gemini-2.5-flash</i> judge	Higher
Diversity	D	1 minus the mean pairwise normalised cosine similarity among question embeddings; measures lexical and semantic variety	Higher
Topic confusion rate	TC	Fraction of generated questions whose embedding aligns more strongly with non-target retrieved chunks than expected target chunks	Lower
Machine answer correctness	AC	Fraction of questions for which an independent verifier selects the same answer option as the generated answer key using source context	Higher
Questions re-turned	Q	Count of valid questions returned per topic; expected value is 6	= 6

Recall@ k and MRR are retrieval-side metrics computed from annotated ground-truth chunk indices in the golden dataset. Semantic grounding, Bloom KL divergence,

LLM judge score, and diversity measure the quality and alignment of the produced question sets. Questions returned is included as a simple completeness check.

For Bloom KL divergence, the target distribution P is obtained from the configured Bloom or difficulty plan, and the observed distribution Q is computed from the labels returned in the generated question set. Each question's `bloom_level` field is self-assigned by the same LLM that generates the question: the structured JSON schema in the prompt instructs the model to output one of the six Bloom levels alongside each question. If the returned value is not a recognised level, the backend maps the `difficulty` field as a fallback (*easy* → *remember*, *medium* → *apply*, *hard* → *analyze*). Bloom KL therefore measures how closely the LLM's self-reported cognitive-level distribution tracks the requested target; the metric's reliability is bounded by the LLM's accuracy at self-classification, which is treated as a limitation in Section 6.4. Lower KL divergence indicates that the generated set follows the requested cognitive mix more closely. In practice, a high KL value usually means that the model over-produced lower-order questions such as *remember* and *understand*, even when the request asked for *apply* or *analyze*.

The semantic grounding metric is interpreted as a proxy, not as a final truth label. It is useful for comparing baselines because the same embedding model and scoring procedure are applied to all outputs. However, a high semantic score does not prove that every answer option is factually correct. This is why the thesis reports both automated metrics and qualitative failure analysis.

The two post-review metrics target failure modes raised during advisor discussion. Topic confusion separates general weak grounding from the more specific case where a generated question drifts toward a different topic within the same document. Machine answer correctness adds an independent answer-key check: the verifier receives the

question, options, and source chunks, but not the original answer key, and its selected option is compared with the generated answer. This does not replace lecturer review, but it gives a second automated signal for hallucination and answer-key risk.

5.3 Baseline Rationale

The three baselines isolate the contribution of each major component. The vanilla baseline represents the simplest practical use of an LLM: sending course content and asking for MCQs without retrieval or pattern conditioning. It establishes the reference point for what a generic LLM workflow can achieve.

The RAG-only baseline adds semantic retrieval but removes exam-pattern conditioning. This baseline tests whether grounding alone improves question quality. If RAG-only improves semantic grounding but still performs poorly on Bloom KL divergence, then source retrieval and cognitive-level control can be understood as separate problems.

The full-system baseline adds both retrieval and pattern conditioning. It therefore represents the proposed QuizGen pipeline. Comparing the full system against RAG-only measures the additional effect of pattern and Bloom control, while comparing it against vanilla measures the combined effect of all proposed components.

All baselines use the same generation model during evaluation. This avoids a common confound in LLM experiments: a stronger model can hide weaknesses in the surrounding pipeline. By holding the model constant, the experiment focuses on whether the architecture improves the output.

5.4 Quantitative Results

Table 14 reports the mean and standard deviation for each metric across three repeated runs.

Table 14: Core baseline comparison (mean $\pm$ std, 3 repeats, 10 topics).

Baseline	SG $\uparrow$	KL $\downarrow$	Judge $\uparrow$	Diversity $\uparrow$	Q ret.
Baseline vanilla	0.7912 ± 0.0048	18.029 ± 1.685	3.783 ± 0.095	0.2152 ± 0.0033	6.0 ± 0.0
RAG only	0.9369 ± 0.0030	11.636 ± 0.960	4.000 ± 0.000	0.1711 ± 0.0031	6.0 ± 0.0
Full system	0.9334 ± 0.0021	3.905 ± 0.782	4.075 ± 0.152	0.1731 ± 0.0032	6.0 ± 0.0

Recall@ k and MRR were 1.0000 ± 0.0000 for all three baselines, confirming that the cosine-similarity retrieval correctly identifies the annotated ground-truth chunk at rank 1 for every topic in this dataset. These metrics therefore do not differentiate the baselines; the meaningful variation lies in the generation-side metrics.

Table ?? reports the post-review retrieval ablation from the cached run 2026-05-04T14:42:34Z. Unlike Table 14, this table compares only variants of the full system and should be read as an ablation of retrieval strategy rather than as a replacement for the core baseline comparison.

Table 15: Core baseline comparison (mean $\pm$ std, 3 repeats, 10 topics).

Baseline	SG $\uparrow$	KL $\downarrow$	Judge $\uparrow$	Diversity $\uparrow$	Q ret.
Baseline vanilla	0.7912 ± 0.0048	18.029 ± 1.685	3.783 ± 0.095	0.2152 ± 0.0033	6.0 ± 0.0
RAG only	0.9369 ± 0.0030	11.636 ± 0.960	4.000 ± 0.000	0.1711 ± 0.0031	6.0 ± 0.0
Full system	0.9334 ± 0.0021	3.905 ± 0.782	4.075 ± 0.152	0.1731 ± 0.0032	6.0 ± 0.0

The ablation gives a mixed but useful signal. Hybrid retrieval slightly improves semantic grounding and Bloom KL compared with dense retrieval, which suggests that exact keyword evidence can complement dense semantic matching. Hybrid reranking slightly lowers topic confusion and raises the judge score, but it also lowers machine answer correctness from 0.9944 to 0.9444. Therefore, reranking is promising but not yet calibrated enough to be treated as a solved retrieval improvement.

Semantic grounding. Adding RAG raises semantic grounding from 0.7912 to 0.9369, a gain of 0.146 points. Retrieved source chunks substantially increase the overlap between generated question content and the lecture material. The full system achieves 0.9334, marginally lower than RAG only, suggesting that pattern conditioning introduces slight lexical variation. That variation can reduce cosine similarity with the raw source text without meaningfully harming grounding quality.

Bloom KL divergence. The full system reduces Bloom KL from 18.029 (vanilla) to 3.905, a reduction of 78.3%. RAG only brings it to 11.636, but the largest gain comes from adding the exam-pattern conditioning that explicitly encodes the target Bloom distribution in the generation prompt. This is the clearest evidence that pattern conditioning helps with cognitive-level alignment.

LLM judge score. The full system achieves the highest judge score (4.075), marginally above RAG only (4.000) and notably above vanilla (3.783). The improvement from vanilla to RAG only is consistent with the observation that grounded questions are judged as more relevant and correct. The small additional gain from RAG only to the full system suggests that pattern conditioning improves clarity and structural quality.

Diversity. Diversity is lowest for RAG-based methods (0.1711 and 0.1731) compared to vanilla (0.2152). This is an expected trade-off. Constraining generation to a small set of retrieved chunks reduces the thematic range of generated questions. For grounded exam generation, the trade-off is acceptable because topic focus is more important than maximising lexical variety.

5.5 Practical Interpretation of Results

I read these quantitative results as evidence for the main thesis claim: question generation quality depends on more than the language model alone. The largest improve-

ment in semantic grounding occurs when retrieval is added. This is expected because the model receives a smaller and more relevant context window rather than a broad document context. In practical terms, this means a lecturer is more likely to receive questions connected to the uploaded lecture topic instead of questions drawn from the model's general knowledge.

The largest improvement in Bloom alignment occurs when pattern conditioning is added to the RAG pipeline. This indicates that retrieved evidence helps the model decide *what* to ask about, while pattern and Bloom instructions help the model decide *how* to ask. The two components therefore serve different roles. Retrieval improves source relevance; pattern conditioning improves assessment-shape alignment.

The slight reduction in semantic grounding from RAG only (0.9369) to full system (0.9334) should not be interpreted as a failure of the proposed method. The difference is small, and the full system gains substantially in Bloom alignment and judge score. A likely explanation is that pattern conditioning encourages the model to rephrase source ideas into more exam-like stems and distractors, which can lower raw embedding similarity while improving pedagogical usefulness.

The diversity metric also requires careful interpretation. In creative text generation, higher diversity is usually desirable. In lecture-grounded MCQ generation, however, too much diversity may indicate that the model is drifting away from the source topic. The lower diversity of RAG-based methods is therefore acceptable when the objective is assessment grounded in a narrow lecture context.

5.6 Failure Analysis

The failure analysis looks at topic-level cases where a baseline exceeds the Bloom KL threshold of 8 or falls below the LLM judge threshold of 4. Table 16 presents five

representative cases that illustrate the main failure patterns.

Table 16: Representative failure cases from the evaluation.

Baseline	Document	Topic	Trigger	Value
Baseline vanilla	os_basics	cpu_scheduling	Bloom mismatch	27.020
Baseline vanilla	it_intro	cloud_models	Bloom mismatch	26.958
Baseline vanilla	db_fundamentals	acid_transactions	Low judge score	3.500
RAG only	os_basics	cpu_scheduling	Bloom mismatch	27.020
Full system	db_fundamentals	acid_transactions	Bloom mismatch	13.469

The full failure analysis shows three patterns. First, Bloom distribution mismatch is the dominant failure mode and is concentrated in the vanilla baseline, which triggers it on nearly every topic that requires higher-order cognitive levels such as *apply* and *analyze*. Without an explicit distribution constraint, the LLM tends to default to recall and understanding questions. Second, RAG only reduces the count of low judge scores to zero, but it does not fully correct Bloom drift because retrieval improves source grounding without specifying the desired cognitive distribution. Third, the full system reduces the total number of failure-triggering topic-level cases from 49 to nine, but two topics remain difficult: *acid_transactions*, where Bloom KL remains above 13 in all three full-system repeats, and *cpu_scheduling*, where Bloom KL falls between 8 and 9 across repeats. Both topics require *apply* or *analyze* questions, which the model has difficulty honoring consistently even when prompted. These cases suggest that prompt-based Bloom conditioning works in aggregate but may need post-generation re-ranking or filtering for high-stakes use.

The post-review failure analysis adds two more signals: topic confusion and answer mismatch. Topic confusion was common even when retrieval recall was perfect, showing that retrieving the expected chunk is not the same as ensuring every generated

question stays concentrated on that chunk. The machine answer verifier found few answer-key mismatches for dense and hybrid full-system runs, but the hybrid-rerank ablation produced a lower answer-correctness score. This supports a conservative interpretation: reranking can help relevance ordering, but answer keys still require separate verification and human review.

5.7 Comparison with Related Work

Table 17 places QuizGen against the systems reviewed in Chapter II using an evidence-based comparison. Rather than repeating the feature checklist of Table 1, the table records whether each system provides a published quantitative metric or empirical evidence for each capability.

Table 17: Evidence-based comparison of QuizGen with related systems.

System	Grounding metric ported	re- ment ated	align- ment evalu- ated	Quiz prac- work- flow	Performance analytics
Generic ChatGPT	No	No	No	No	No
E-QGen [31]	No	No	No	No	No
BUE AQG [32]	User study (80% satis- faction)	No	No	No	No
COLING 2025 [30]	BLEU/ROUGE only	No	No	No	No
QuizGen (this the- sis)	Semantic grounding score (0.93)	Bloom KL divergence (3.91)	Yes (attempt tracking)	Yes (confi- dence trend)	

Against generic LLM prompting, QuizGen adds source retrieval and a grounding evaluation mechanism that surfaces weak questions before publication. Against RAG-only generation, it adds exam-pattern conditioning and Bloom-level control, reducing

KL divergence from 11.636 to 3.905. Unlike static question generation tools such as E-QGen and BUE AQG, QuizGen also connects generated questions to quiz practice, attempt tracking, and confidence trend analytics.

5.8 Practical Implications for Lecturers and Students

For lecturers, the main implication is that LLM-based question generation should be treated as assisted drafting, not automatic exam authoring. RAG and pattern conditioning improve grounding and alignment, but the failure analysis shows that Bloom mismatch can still occur. A lecturer should therefore review generated questions, inspect low-grounding items, and adjust or remove weak questions before using the output in formal assessment.

For students, the system is most useful as a formative practice tool. The questions are generated from lecture material, the quiz interface provides immediate scoring, and the attempt review page shows explanations after submission. The confidence trend can help a student see whether repeated practice on the same topic is improving performance. This closes the loop between generated content and learning feedback.

For system designers, the results suggest that a useful educational AI application should combine generation with workflow support. A standalone prompt can create questions, but it does not manage source files, preserve configuration, track attempts, evaluate grounding, or show progress over time. In QuizGen, these surrounding features are not merely interface details; they are part of what makes generated questions usable in an educational setting.

The results also mark the boundary of automation. QuizGen can score grounding and summarize trends, but it cannot fully judge pedagogical appropriateness, cultural context, course emphasis, or distractor fairness. Those decisions remain part of the

lecturer's professional role.

5.9 Threats to Validity

Several limitations affect the interpretation of the evaluation results.

Dataset scale. The evaluation covers ten topics across four documents in two languages. This scale is sufficient for a thesis-level proof of concept but is not large enough to make broad claims about generalisability across all subject domains or languages.

LLM judge bias. The judge model (*gemini-2.5-flash*) assigns scores based on its own training distribution. It may be systematically lenient or strict across topics and may correlate with the generation model through shared training data. The strict rubric and the instruction to use the full scoring range mitigate but do not eliminate this bias.

Grounding score as a proxy. Semantic grounding measures cosine overlap between question embeddings and source chunk embeddings. A high score indicates topical alignment but does not guarantee factual correctness. A well-grounded question could still contain an incorrect answer option that is semantically close to the source text.

Retrieval ceiling effect. Recall@ k and MRR were 1.0 for all baselines because the golden dataset is small and each topic has a precisely annotated ground-truth chunk that the retriever always ranks first. On a larger or noisier document collection, retrieval metrics would likely show meaningful differences between baselines.

Single-model evaluation. All three core baselines use the same generation model. Results may differ with other models; multi-model comparison baselines are configured in `eval/config.yaml` but their results were not collected for this thesis and are left as future work.

Local deployment only. The system has not been tested under concurrent load or with a production database. Latency and throughput characteristics may differ

significantly in a classroom deployment.

5.10 Summary of Findings

The evaluation leads to three main findings. First, adding RAG to vanilla LLM generation substantially improves semantic grounding, from 0.7912 to 0.9369, and LLM judge quality, from 3.783 to 4.000. This supports the premise that source retrieval is necessary for lecture-grounded MCQ generation. Second, adding exam-pattern conditioning to the RAG pipeline reduces Bloom KL divergence by 66.5%, from 11.636 to 3.905, showing that few-shot pattern injection can improve cognitive-level alignment without model fine-tuning. Third, the full system achieves the best LLM judge score, 4.075, while maintaining high semantic grounding, 0.9334. The main remaining limitation is that Bloom distribution control is not perfectly consistent at the individual topic level, especially for higher-order cognitive levels, so lecturer review before formal exam use remains necessary.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 Conclusion

In this thesis I designed, built, and evaluated QuizGen, a pattern-aware retrieval-augmented generation platform for producing grounded multiple-choice questions from lecture materials. The work addresses four limitations identified in the reviewed literature: unsupported LLM generation, weak alignment with lecturer exam patterns, limited Bloom-level control, and the lack of an integrated quiz practice workflow.

In the implemented workflow, PDF, DOCX, and PPTX lecture documents are converted into overlapping word chunks, embedded through the configured embedding provider, and searched with cosine similarity during generation. Retrieved chunks and optional exam-pattern examples are then placed into the prompt, allowing the LLM provider to return structured MCQs with grounding status for each item. Around this generation pipeline, the web application supports question review, quiz practice, attempt review, history, usage tracking, and dashboard visualization.

The evaluation used ten topics in English and Vietnamese with three repeated runs per baseline. RAG improved semantic grounding from 0.7912 to 0.9369 over vanilla LLM prompting. Pattern-aware generation reduced Bloom KL divergence by 66.5% compared with RAG only, from 11.636 to 3.905. The full system also achieved the highest LLM judge score, 4.075, while returning all requested questions in every run. Taken together, these results support the claim that retrieval, exam-pattern conditioning, and Bloom control are useful components for lecture-grounded MCQ generation. They

do not remove the need for lecturer review before high-stakes use.

6.2 Contributions

The main contributions are:

1. A pattern-aware RAG pipeline that combines semantic retrieval, few-shot exam-pattern conditioning, and Bloom-level distribution control without model fine-tuning.
2. A grounding proxy based on normalized cosine similarity between generated question content and source material, extended with topic-confusion and answer-key verification signals, exposed as a review aid rather than as a replacement for lecturer judgment.
3. A complete web workflow for document upload, MCQ generation, review, quiz practice, attempt review, history, usage tracking, and dashboard visualization.
4. A reproducible evaluation setup with fixed baselines, repeated runs, structured result files, checkpoint logs, failure analysis, and a post-review retrieval ablation.
5. A provider-routing layer that supports Groq, Gemini, OpenRouter, and local Ollama fallback options while logging provider usage and token counts.

6.3 Achievement of Objectives

Table 18 summarizes how the implementation and evaluation address the objectives introduced in Chapter I.

Table 18: Achievement of thesis objectives.

Objective	Achievement in QuizGen
Process PDF, DOCX, and PPTX lecture materials	Implemented document upload, extraction, chunking, and storage through the document processor service.
Apply retrieval-augmented generation	Implemented embedding generation, cosine-similarity chunk retrieval, prompt context injection, and post-review hybrid/rerank ablation modes.
Incorporate exam-pattern conditioning	Implemented pattern creation, pattern storage, and few-shot prompt injection during generation.
Generate structured MCQ outputs	Implemented JSON-based question generation with answer options, correct answers, explanations, difficulty labels, and Bloom labels.
Provide grounding evaluation	Implemented semantic grounding scores, topic-confusion analysis, answer-key verification, and review-facing grounding status labels.
Implement quiz practice and analytics	Implemented quiz attempt submission, attempt review, confidence trend, Bloom breakdown, history, and usage dashboards.
Evaluate against baselines	Implemented a reproducible evaluation runner and reported vanilla, RAG-only, full-system, and post-review retrieval-ablation comparisons.

At prototype scale, the objectives were achieved. The implementation covers the full workflow, and the evaluation shows measurable improvements from the proposed pipeline components. Production readiness and fully automated exam authoring remain outside the scope of this thesis.

6.4 Limitations

Several limitations should be noted. First, QuizGen focuses on MCQ generation; open-ended, short-answer, and essay questions are outside the current scope. Second, the evaluation uses a thesis-scale dataset of ten topics across four documents, so the results may not generalize to all subjects, languages, or document formats. Third, testing was carried out in a local prototype setting rather than in a classroom deployment with many concurrent users. Fourth, the grounding score is a semantic proxy: it can highlight source-support risk but cannot prove factual correctness. The post-review answer verifier reduces this risk but does not remove the need for lecturer checking. Fifth, the current generation API is single-document: although the batch workflow accepts multiple document identifiers, it creates independent quizzes per document rather than one merged quiz with a controlled 50/50 or proportional split. Sixth, the evaluation uses LLM-based judges and verifiers rather than a panel of domain-expert lecturers, so human assessment of relevance, distractor quality, answer correctness, and pedagogical usefulness remains necessary.

6.5 Future Work

Future work can extend QuizGen in several directions. True multi-document retrieval would allow quizzes to be generated from an entire module rather than one lecture file, with either length-weighted or user-specified proportional sampling and topic-confusion guard rails. Retrieval can also be strengthened through better keyword expansion, document-schema-aware signals such as headings or slide numbers, and more calibrated reranking.

Generation-side future work includes verifying whether difficulty labels match

the actual cognitive demand of each question, extending a main idea across Bloom levels such as definition-to-apply transformations, and adding a pattern-variation engine that can safely create same-form, changed-number, fill-in-the-blank, and tricky-distractor variants. These extensions require strong answer verification because uncontrolled variation can easily break correctness.

Evaluation and deployment also need expansion. A larger golden dataset across more subjects, document types, and languages would strengthen the empirical evaluation, while human lecturer evaluation would validate relevance, pedagogical value, answer correctness, and distractor quality. Reinforcement learning from human feedback, as used in instruction-tuned LLMs [28], is a longer-term direction for learning from lecturer preferences, but it is outside the scope of this prototype. PDF export, spaced repetition, a production-grade database, asynchronous generation jobs, concurrency testing, and formal evaluation of local LLM fallbacks remain practical deployment improvements.

LIST OF REFERENCES

- [1] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Nee-lakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [2] Xu, X., Gan, Y., Zhang, X., Shi, Y., Li, Y., Dong, Y., and Tang, J. 2024. Large language models for education: A survey and outlook. *arXiv preprint arXiv:2403.18105*.
- [3] Xu, X., Li, Y., Dong, Y., Zhang, X., Shi, Y., Gan, Y., and Tang, J. 2024. Large language models for education: A survey. *arXiv preprint arXiv:2405.13001*.
- [4] Rasul, T., Nair, S., Kalendra, D., Robin, M., de Oliveira Santini, F., Ladeira, W. J., Sun, M., Day, I., Rather, R. A., and Heathcote, L. 2024. The role of ChatGPT in higher education: Benefits, challenges, and future research directions. *Frontiers in Education*, 8, 1–15.
- [5] Pham, A., Dobbins, C., and Ning, H. 2023. Evaluating ChatGPT for question generation from passage context. In *Proceedings of the IEEE International Conference on Big Data (BigData)*, pp. 4105–4113. IEEE.

- [6] Danyaro, K. U., Liew, M. S., Ahmed, A., and Hussain, A. 2024. Hallucinations in large language models for education: Challenges and mitigation. *International Journal of Technology in Learning and Education*.
- [7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [8] Es, S., James, J., Anke, L. E., and Schockaert, S. 2023. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.
- [9] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., and Schmidt, D. C. 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382*.
- [10] Weng, L. 2023. Prompt engineering. *Lil'Log blog*. [Online]. Available: <https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/>. [Accessed: May 1, 2026].
- [11] Bloom, B. S., Engelhart, M. D., Furst, E. J., Hill, W. H., and Krathwohl, D. R. 1956. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. David McKay, New York.
- [12] Anderson, L. W., Krathwohl, D. R., Airasian, P. W., Cruikshank, K. A., Mayer, R. E., Pintrich, P. R., Raths, J., and Wittrock, M. C. 2001. *A Taxonomy for Learning*.

ing, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, New York.

- [13] Anderson, J. R., Boyle, C. F., and Reiser, B. J. 1985. Intelligent tutoring systems. *Science*, 228(4698), 456–462.
- [14] Springer Nature. 2022. Automated essay scoring systems. In *Encyclopedia of Education and Information Technologies*. [Online]. Available: https://link.springer.com/rwe/10.1007/978-981-19-2080-6_59. [Accessed: May 1, 2026].
- [15] Khan Academy. *Khanmigo: Khan Academy's AI-powered teaching assistant and tutor*. [Online]. Available: <https://www.khanmigo.ai/>. [Accessed: May 1, 2026].
- [16] Duolingo Team. 2023. Introducing Duolingo Max, a learning experience powered by GPT-4. [Online]. Available: <https://blog.duolingo.com/duolingo-max/>. [Accessed: May 1, 2026].
- [17] Robertson, S., and Zaragoza, H. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [18] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP 2020*, pp. 6769–6781.
- [19] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. 2009. Reciprocal rank fusion

outperforms Condorcet and individual rank learning methods. In *Proceedings of SIGIR 2009*, pp. 758–759.

- [20] Nogueira, R., and Cho, K. 2019. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- [21] Izacard, G., and Grave, E. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of EACL 2021*, pp. 874–880.
- [22] Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., Zettlemoyer, L., and Yih, W. 2023. REPLUG: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*.
- [23] Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. 2023. ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*.
- [24] Pan, L., Lei, W., Chua, T.-S., and Kan, M.-Y. 2019. Recent advances in neural question generation. *arXiv preprint arXiv:1905.08949*.
- [25] Kurdi, G., Leo, J., Parsia, B., Sattler, U., and Al-Emari, S. 2020. A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education*, 30, 121–204.
- [26] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., and Zhou, D. 2022. Chain-of-thought prompting elicits reasoning in large language

- models. In *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- [27] Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- [28] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 35, 27730–27744.
- [29] Alkhazaey, S., Grasso, F., Payne, T. R., and Tamma, V. 2025. Text-based question generation using transformers and large language models: A systematic literature review. *IEEE Transactions on Learning Technologies*, 18, 1–20.
- [30] Rodríguez-García, M., Sánchez-Rada, J. F., and Iglesias, C. A. 2025. Automated MCQ generation and validation using large language models. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, pp. 1842–1853.
- [31] Hadifar, A., Bitew, S. K., Deleu, J., Develder, C., and Demeester, T. 2024. E-QGen: Educational lecture abstract-based question generation system. *arXiv preprint arXiv:2404.13547*.
- [32] Shaban, W. M., Elhoseny, M., and Hassanien, A. E. 2024. Automatic question generation using GPT, T5, and BERT for educational assessment. *Journal of King*

Saud University – Computer and Information Sciences, 36(3), 101983.

- [33] Quillionz. *Quillionz: AI-powered question generator*. [Online]. Available: <https://quillionz.com/>. [Accessed: May 1, 2026].
- [34] Questgen. *Questgen: AI quiz generator*. [Online]. Available: <https://www.questgen.ai/>. [Accessed: May 1, 2026].
- [35] Google AI for Developers. *Gemini API embeddings documentation*. [Online]. Available: <https://ai.google.dev/gemini-api/docs/embeddings>. [Accessed: May 1, 2026].

APPENDICES

APPENDIX A: EXAMPLE GENERATED MCQS

The following examples illustrate the structure of a generated question record. They are not synthetic examples; they are taken from the local SQLite database backend/data/quizgen.db, generation id 19, titled *Database Fundamentals - Demo Confidence Quiz*. The generation was completed on 2026-04-29T04:10:12.402146+00:00, used provider groq, prompt version v1, and recorded token usage of 5,246. The generation contains five MCQs.

1. **Question.** What is the main problem with managing data resources in a traditional file environment?

Options. A. Data redundancy and inconsistency; B. Lack of data sharing and availability; C. Poor security; D. All of the above.

Correct answer. D. All of the above.

Explanation. According to the source material, the traditional file environment has several problems, including data redundancy, data inconsistency, program-data dependence, lack of flexibility, poor security, and lack of data sharing and availability.

Difficulty and Bloom level. Easy; Remember.

2. **Question.** What is the primary function of a database management system (DBMS)?

Options. A. To provide a user interface for data entry; B. To interface between

applications and physical data files; C. To provide data storage and retrieval; D. To perform data analysis and reporting.

Correct answer. B. To interface between applications and physical data files.

Explanation. The source material states that a DBMS interfaces between applications and physical data files, separating logical and physical views of data.

Difficulty and Bloom level. Medium; Understand.

3. **Question.** What is the purpose of normalization in database design?

Options. A. To reduce data redundancy and improve data integrity; B. To improve data security; C. To increase data storage capacity; D. To enhance data retrieval performance.

Correct answer. A. To reduce data redundancy and improve data integrity.

Explanation. The source material explains that normalization is the process of streamlining complex groupings of data to minimize redundant data elements and awkward many-to-many relationships.

Difficulty and Bloom level. Medium; Apply.

4. **Question.** What is the primary benefit of using a relational DBMS?

Options. A. Improved data security; B. Increased data storage capacity; C. Enhanced data retrieval performance; D. Ability to represent data as two-dimensional tables.

Correct answer. D. Ability to represent data as two-dimensional tables.

Explanation. The source material states that a relational DBMS represents data as two-dimensional tables, with each table containing data on an entity and its attributes.

Difficulty and Bloom level. Easy; Remember.

5. **Question.** What is the purpose of a data warehouse?

Options. A. To store current and historical data from many core operational transaction systems; B. To provide data storage and retrieval; C. To perform data analysis and reporting; D. To interface between applications and physical data files.

Correct answer. A. To store current and historical data from many core operational transaction systems.

Explanation. The source material explains that a data warehouse stores current and historical data from many core operational transaction systems, consolidating and standardizing information for use across the enterprise.

Difficulty and Bloom level. Medium; Understand.

APPENDIX B: EVALUATION CONFIGURATION

The final evaluation configuration is summarized below. The controlled comparison uses three core baselines, and the post-review ablation compares three full-system retrieval modes. Both result sets are reported as cached runs in Chapter V.

Table 19: Evaluation configuration summary.

Item	Value
Dataset	eval/datasets/golden.json
Documents	IT Fundamentals, DB Fundamentals, OS Principles, Network Security
Topics	10 topics, including English and Vietnamese topics

Repeats	3 per baseline
Questions per topic	6
Prompt version	v1
Core baselines	baseline_vanilla, baseline_rag_only, full_system
Retrieval ablation	full_system, full_system_hybrid, full_system_hybrid_rerank
Generation model	meta-llama/llama-4-scout-17b-16e-instruct
Judge and verifier models	Core run: gemini-2.5-flash; post-review ablation: openrouter/auto-free
Embedding model	Core run: gemini-embedding-001; post-review ablation: nvidia/llama-nemotron-embed-v1-1b-v2:free
Random seed	42
Output files	Latest snapshots: comparison.csv, details.csv, failure_analysis.md; history: runs.csv, history.md; checkpoint JSONL files

The core reproduction command is:

The core reproduction command is:

```
set PYTHONPATH=backend && python eval/run_eval.py --config eval/config.yaml ^
--baselines baseline_vanilla baseline_rag_only full_system
```

The post-review retrieval ablation command is:

```
set PYTHONPATH=backend && python eval/run_eval.py --config eval/config.yaml ^
--baselines full_system full_system_hybrid full_system_hybrid_rerank
```

APPENDIX C: FAILURE ANALYSIS TABLE

The failure-analysis report flags topic-level outputs when Bloom KL divergence exceeds the configured threshold or the LLM judge score falls below the configured quality threshold. The representative cases in Table 20 are used to support the discussion in Chapter V.

Table 20: Representative evaluation failures.

Baseline	Document	Topic	Trigger	Value
Baseline vanilla	os_basics	cpu_scheduling	Bloom mismatch	27.020
Baseline vanilla	it_intro	cloud_models	Bloom mismatch	26.958
Baseline vanilla	db_fundamentals	acid_transactions	Low judge score	3.500
RAG only	os_basics	cpu_scheduling	Bloom mismatch	27.020
Full system	db_fundamentals	acid_transactions	Bloom mismatch	13.469

The dominant failure type is Bloom distribution mismatch. This indicates that the LLM can follow source content while still failing to produce the requested cognitive-level mix. The full system reduces the number of such failures, but it does not eliminate them completely.

APPENDIX D: API ENDPOINT LIST

Table 21 lists the main API groups implemented in the backend.

Table 21: Main API endpoint groups.

Group	Representative endpoint	Purpose
-------	-------------------------	---------

Health	GET /api/health	Verify backend availability
Auth	POST /api/auth/register	Register a user account
Auth	POST /api/auth/login	Issue a JWT token
Auth	GET /api/auth/me	Return current authenticated user
Documents	POST /api/documents/upload	Upload and process lecture material
Documents	GET /api/documents/	List user-owned documents
Patterns	POST /api/patterns/	Create an exam pattern
Patterns	GET /api/patterns/	List saved patterns
Generations	POST /api/generations/	Generate questions
Generations	GET /api/generations/	List saved generations
Quiz	POST /api/quiz/submit	Submit quiz answers
Quiz	GET /api/quiz/attempts	List saved attempts
Dashboard	GET /api/dashboard/trend	Return confidence trend data
Usage	GET /api/usage/	Return provider usage statistics
Evaluation	GET /api/eval/latest	Return latest evaluation result
Batch	POST /api/batch/	Start a batch generation job

APPENDIX E: ADDITIONAL SCREENSHOTS

The screenshot assets used in Chapter IV are stored under `doc/screenshots`. They document the implemented Atelier interface and provide visual evidence for the thesis demonstration. The final capture pass used light mode to improve legibility in the printed thesis.

Table 22: Screenshot asset inventory.

File	Shown feature
01-workflow-source.png	Document upload and source selection
02-workflow-pattern.png	Pattern and difficulty configuration
03-workflow-generate-title.png	Quiz title entry before generation
04-workflow-review.png	Generated question review
05-quiz-practice.png	Timed quiz practice screen
06-attempt-review.png	Attempt review with explanations
07-history-rename.png	Generation history and rename workflow
08-dashboard-confidence-trend.png	Per-quiz confidence trend chart
09-evaluation-dashboard.png	Evaluation dashboard
10-usage-dashboard.png	Provider usage dashboard
11-settings-config.png	Configuration and fallback settings
12-batch-workflow.png	Batch generation workflow

APPENDIX F: SETUP AND RUN INSTRUCTIONS

The application can be started locally on Windows with the provided batch script:

```
start-all.bat
```

The script starts the backend on `http://localhost:8000` and the frontend on `http://localhost:3000`. After startup, the backend health endpoint can be checked with:

```
GET http://localhost:8000/api/health
```

For containerized execution, the application can be started with Docker Compose:

```
docker compose up --build
```

The recommended demo sequence is:

1. Log in with a prepared demo account.
2. Upload or select a lecture document.
3. Select either no pattern or a saved exam pattern.
4. Configure question count, language, and difficulty distribution.
5. Generate a quiz and inspect the review page.
6. Start the quiz, submit answers, and open the attempt review page.
7. Open the dashboard to show confidence trend and Bloom breakdown.
8. Open the usage dashboard to show provider usage and fallback status.
9. Open the evaluation dashboard to show the cached baseline results.

If live API quota is unavailable during demonstration, the prepared screenshots, cached evaluation outputs, and saved generation records can be used to demonstrate the full workflow without relying on a new LLM call.